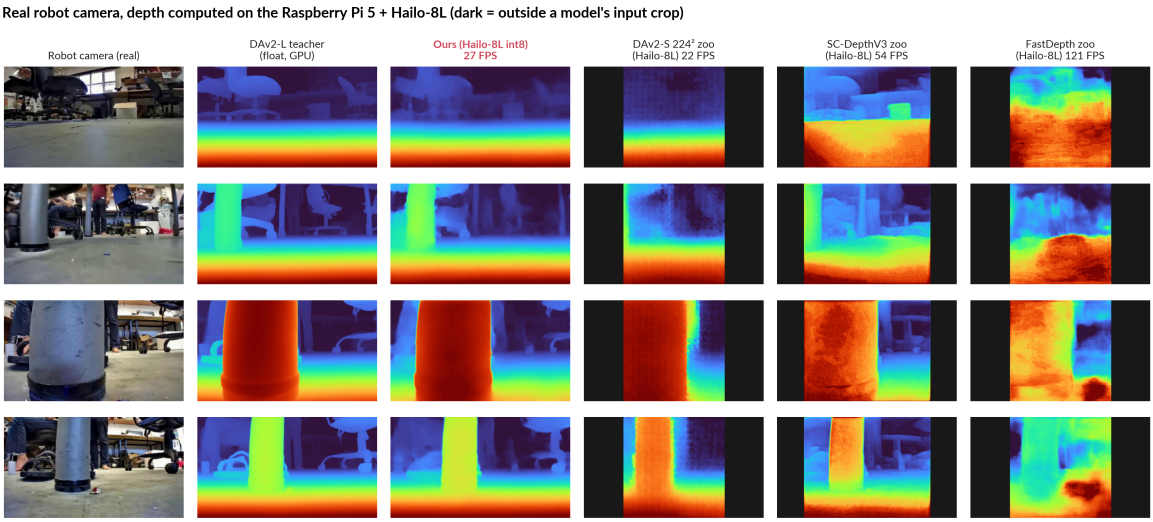
Seeing to Drive on a Single Board

A distilled, attention-free monocular depth network on a Raspberry Pi 5 + Hailo-8L, and the onboard autonomy it enables: point clouds, obstacle avoidance, global path planning and SLAM for a small rover

Itai M.

Thesis project DANIEL — Depth Anything Navigation In Exteriors, Learned



Version of 12 November 2026. Every number in this document is a measurement from the project's own experiments; the scripts that produced each table and figure are listed in Appendix A.

Small ground robots rarely carry LiDAR or stereo; a single camera and a few watts of compute are what they can afford. Monocular depth foundation models such as Depth Anything V2 now produce remarkably good relative depth, but they are vision transformers sized for GPUs. This thesis takes one of them to a Raspberry Pi 5 with a 13-TOPS Hailo-8L neural accelerator and builds a complete onboard autonomy stack on top of it.

We first show, by measurement on the chip, *why* transformer depth breaks on this accelerator: the Hailo-8L executes attention in int8, and at 449 tokens a diffuse attention weight falls below a single quantisation step. Our own int8 build of Depth Anything V2-Small, and Hailo's official one, lose a large part of the scene. We therefore distil Depth Anything V2-Large into an *attention-free* ResNet-34 U-Net (24.5 M parameters) and fine-tune it on rendered ground truth. On the real Hailo-8L it runs at **27.1 FPS at 288×512**, its int8 output matches its own float twin at Pearson **0.99993**, and on 400 held-out simulated frames with metric ground truth it is the most accurate of every depth network we ran on the chip (AbsRel 0.269 overall versus 0.354–0.530 for every other build, including the vendor's), while agreeing with its 335 M-parameter teacher at $r = 0.995$ on real robot video (on the real 1,000-point benchmark below it trails that teacher by 0.085 AbsRel). On 1,000 stereo-verified points of a real outdoor drive it is not resolvably different from the vendor's build (on the three-matcher set the paired interval bounds any advantage of ours at 0.060 AbsRel, below the 0.085 measured in simulation on a different metric; in relative terms only the exact block test excludes it, by one point (bound 23 % against 24 %), and on the two-matcher set neither does), and its simulated advantage on obstacles is not shown there (a post-hoc, exploratory lean on log error: block $p = 0.046$, 0.046–0.14 after a Holm correction depending on the test family).

Relative depth has no metric scale, so we range obstacles by *ground-contact geometry* instead: the image row where an object meets the floor fixes its distance from the camera height alone. On this scan we build a hit-count map, hybrid-A* global planning, pure-pursuit control, and a SLAM front end (ground-plane visual odometry, optional wheel odometry, and an image-based slip guard). The whole pipeline fits the Pi: 35 ms on the NPU in parallel with ≈49 ms on the CPU (stage medians, deployed code), with planning in an asynchronous worker.

We evaluate the system with the real Pi and Hailo-8L in a real-time hardware-in-the-loop simulation: a Gaussian-splat reconstruction of a real yard in Isaac Sim, eleven kinds of realistic obstacles with honest colliders, ten scenes and at least five seeds per configuration, scored on the car's real footprint. The deployed stack reaches the goal without any contact in **7.9 ± 1.1 of 10** scenes with simulator pose and **7.7 ± 0.7** with its own SLAM pose (simulated wheel encoder and gyro; ten seeds each), **8.2 ± 0.4** with the real board in the loop (five seeds), against **9.2** for the same real-time stack fed exact obstacles and 9.8 for an oracle. Camera-only odometry reaches 2.6. We report the full error budget, more than a dozen negative results, and what remains between this system and a perfect score: obstacle bases lower than the scan's 5 cm threshold, and exploration around obstacles the camera has not yet seen.

Contents

Abstract 2

1 Introduction 5
- 1.1 The problem in one paragraph . . . 5
- 1.2 Contributions . . . 6
- 1.3 Headline numbers . . . 7
- 1.4 Outline . . . 7

2 Background and related work 8
- 2.1 Monocular depth: from regression to foundation models . . . 8
- 2.2 Depth on small computers . . . 8
- 2.3 Quantising transformers . . . 9
- 2.4 Navigating with predicted depth . . . 9
- 2.5 SLAM . . . 9
- 2.6 Simulation and hardware in the loop . . . 9
- 2.7 The state of the art in October 2026 . . . 10
  - 2.7.1 Depth foundation models now carry the camera . . . 10
  - 2.7.2 Small depth: everyone now distils, few report int8 . . . 10
  - 2.7.3 Stereo as the reference, not the competitor . . . 11
  - 2.7.4 2D SLAM: a textbook algorithm with an unusual sensor . . . 11
  - 2.7.5 Visual and learned SLAM . . . 12
  - 2.7.6 Monocular depth as a range sensor . . . 12
  - 2.7.7 Long-term operation . . . 12
  - 2.7.8 How our evaluation follows the literature . . . 13
  - 2.7.9 What remains ours . . . 13
- 2.8 Positioning . . . 13

3 An attention-free depth student for the Hailo-8L 15
- 3.1 Platform . . . 15
- 3.2 Why transformer depth breaks on this chip . . . 15
- 3.3 The student . . . 16
- 3.4 Compilation and on-chip fidelity . . . 17

4 Benchmark: depth on the edge 19
- 4.1 Protocol . . . 19
- 4.2 Results . . . 20
- 4.3 What the benchmark does and does not show . . . 21

5 From relative depth to metric space 23
- 5.1 Ground-contact geometry . . . 23
- 5.2 The scan . . . 23

5.3 Point clouds . . . 25
5.4 The map . . . 25
5.5 The measured limit: low bases . . . 25

6 Onboard autonomy: planning, control and SLAM 27
6.1 Global planning: hybrid A* on sensed obstacles . . . 27
6.2 Control and safety behaviours . . . 28
6.3 SLAM . . . 28
6.4 The real-time budget on the Pi . . . 29

7 Hardware-in-the-loop evaluation 31
7.1 Set-up . . . 31
7.2 Results . . . 32
7.3 Failure analysis . . . 33
7.4 What did not help . . . 35
7.5 Statistics . . . 36

8 The real world: one drive, every sensor, every method 37
8.1 The recording instrument . . . 37
8.2 Which depth is better: a 1000-point benchmark . . . 38
8.3 Same poses, swap the depth . . . 42
8.4 SLAM on the real drive . . . 42
8.5 SLAM with ground truth: which depth is better inside SLAM? . . . 44
8.6 Objects at a fixed distance and speed . . . 47

9 Discussion 49
9.1 What is new . . . 49
9.2 Limits, stated plainly . . . 50
9.3 Threats to validity . . . 50

10 Conclusion and future work 51
10.1 Closing the gap . . . 51
10.2 Before the first autonomous drive . . . 51

Bibliography 53

A Reproduction 66
A.1 Depth student . . . 66
A.2 Benchmark (Chapter 4) . . . 66
A.3 Navigation stack and HITL . . . 67
A.4 Commands . . . 67

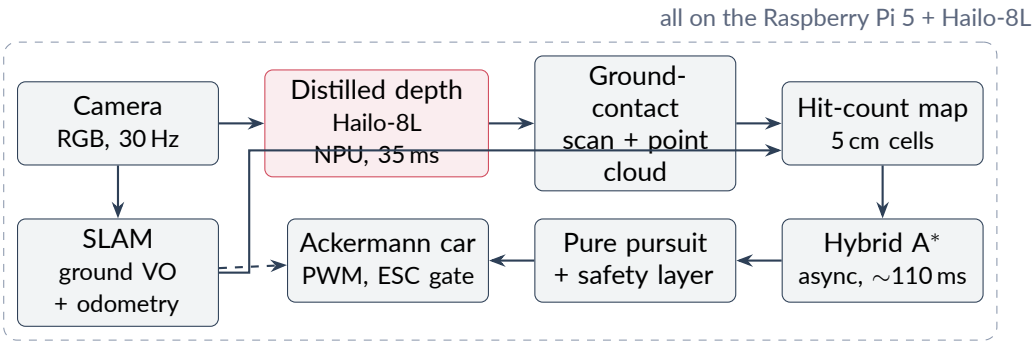
Chapter 1

Introduction

A small rover that has to find its way through a yard full of chairs, cones, crates and people needs to know where things are. The classical answer is a range sensor: a 2D or 3D LiDAR, a stereo pair, or a time-of-flight camera. Each costs money, power, mass and integration effort, and each has blind spots of its own. A planar LiDAR does not see a table top or a low curb; stereo needs texture and a baseline that a palm-sized vehicle does not have. A single RGB camera is the cheapest and lightest sensor a robot can carry, and it sees all of these things. What it does not give directly is *distance*.

In the last three years monocular depth estimation has changed character. Networks trained on tens of millions of images and distilled from synthetic ground truth—MiDaS [108], Depth Anything [152], Depth Anything V2 [153], Depth Pro [9] and, most recently, Depth Anything 3 [73]—now predict relative depth that is sharp, stable and general enough to drive with. They are, however, vision transformers sized for data-centre GPUs: Depth Anything V2-Large has 335 M parameters and, even on the RTX 5090 used in this work, takes 175 ms per frame at its native resolution. A small robot has a few watts.

This thesis asks how much of that capability survives on the cheapest serious edge platform available today, a Raspberry Pi 5 with the 13-TOPS Hailo-8L accelerator of the Raspberry Pi AI Kit [109], and what a rover can do with it. The answer we give is a complete, measured system:



1.1 The problem in one paragraph

Running Depth Anything V2 on the Hailo-8L sounds like a compilation task. It is not. The accelerator executes every layer in integer arithmetic, attention included, and attention over

a few hundred image tokens produces weights so small that int8 cannot represent them. The obvious build loses large parts of the scene, and so does the vendor's own build of the same network (Chapters 3 and 4). Even with a faithful depth map, the output is *relative*: correct up to an unknown scale and shift, so it cannot by itself tell a planner that a chair is 1.2 m away. And even with metric obstacles, a car needs a map, a planner, a controller and an estimate of its own position, all inside the same few watts and the same 50-millisecond control period.

1.2 Contributions

C1 A measured explanation of why transformer depth fails on the Hailo-8L, and an attention-free student that does not. We show on the chip that int8 attention erases obstacles in Depth Anything V2-Small builds (ours and the vendor's), and we distil Depth Anything V2-Large into a 24.5 M-parameter ResNet-34 U-Net with no attention at all. Its int8 output on the real Hailo-8L matches its float twin at Pearson 0.99995 (p5 0.99897, 500 frames), and it runs at 27.1 FPS at 288×512—1.7× the pixels of our Depth Anything V2-S build at 1.6× its speed (Chapter 3).

C2 The first head-to-head benchmark of every depth network we could run on the Hailo-8L, on 400 held-out frames with metric ground truth and 100 real robot frames. On held-out simulated frames the student is the most accurate on the chip by a wide margin (AbsRel 0.269 against 0.354–0.530) and, in simulation, stays within a few points of its 335 M-parameter teacher (Chapter 4); on stereo-verified real outdoor points it is not resolvably different from the vendor's Depth Anything V2-S and trails its teacher by 0.085 AbsRel (Section 8.2).

C3 Scale-free metric ranging from relative depth. A ground-contact obstacle scan turns the student's relative depth into a metric laser-like scan using only the camera height, with three fixes we found by failure analysis: a horizon-anchored floor model, a scale-free vertical-face detector and a floating-overhang detector (Chapter 5).

C4 A complete onboard autonomy stack within the Pi's budget: point clouds, a hit-count map, hybrid-A* global planning with a safety cascade, pure-pursuit control and a SLAM front end with an encoder-gated slip guard. All of it is measured on the robot computer: NPU 35 ms in parallel with about 49 ms of CPU (Chapter 6).

C5 A real-time hardware-in-the-loop evaluation with an explicit error budget. The real Pi and Hailo-8L drive a simulated Traxxas through a Gaussian-splat reconstruction of a real yard. There are ten scenes, eleven obstacle types with honest colliders, at least five seeds per configuration, and contacts are scored on the car's real footprint. Oracle and exact-obstacle ceilings separate perception, planning and localisation error (Chapter 7).

C6 An honest record. More than a dozen interventions that did *not* help are reported with the same rigour as those that did, together with the measured limits that remain (Chapter 9).

1.3 Headline numbers

On the Raspberry Pi 5 + Hailo-8L: 27.1 FPS depth at 288×512, int8 = float (r = 0.99993), AbsRel 0.269 on held-out metric GT (next best on the chip: 0.354). In the real-time yard suite: 7.9 ± 1.1 of 10 scenes reached without contact with simulator pose and 7.7 ± 0.7 on the robot's own SLAM pose (ten seeds each), 8.2 ± 0.4 with the real board in the loop, against 9.2 for the same stack fed exact obstacles.

1.4 Outline

Chapter 2 places the work among recent monocular depth, edge-inference and depth-based navigation research. Chapter 3 explains the int8 attention failure and the student that avoids it. Chapter 4 benchmarks it against every alternative on the chip and against the float foundation models. Chapter 5 turns relative depth into metric space. Chapter 6 describes the planner, controller and SLAM and their real-time budget on the Pi. Chapter 7 evaluates the whole system in the loop. Chapter 9 discusses what is new, what is not, and what remains. Chapter 10 concludes and lists what the first real drive needs.

Chapter 2

Background and related work

2.1 Monocular depth: from regression to foundation models

Learning depth from one image goes back to multi-scale regression on indoor and driving datasets [22]. Two ideas made it general. MiDaS [108] trained on a mixture of datasets with a *scale- and shift-invariant* loss, predicting *relative* inverse depth that transfers zero-shot. DPT [107] replaced the convolutional encoder with a vision transformer. Depth Anything [152] scaled this recipe to 62 M unlabelled images through pseudo-labelling, and Depth Anything V2 [153] trained a large teacher on purely synthetic, precise ground truth and distilled it through pseudo-labelled real images into students of 25 M (Small), 98 M (Base) and 335 M (Large) parameters. Its detail and robustness are why it is our teacher. A parallel line predicts *metric* depth directly, by conditioning on or estimating the camera: Metric3D v2 [41], UniDepth [101] and Depth Pro [9], which produces a 2.25-megapixel metric map in 0.3 s on a GPU. Generative models repurposed for depth, such as Marigold [52], trade speed for detail. Most recently, Depth Anything 3 [73] uses a single plain transformer with a depth-ray target to recover consistent geometry from any number of views; Video Depth Anything [12] adds temporal consistency to Depth Anything V2; and Distill Any Depth [37] shows that distillation choices alone, notably avoiding global normalisation of noisy pseudo-labels, produce a stronger estimator. A 2026 survey [77] traces this whole arc. Every one of these models is a transformer, and none was designed for an integer-only accelerator.

2.2 Depth on small computers

FastDepth [144] showed real-time depth on a Jetson TX2 with a pruned MobileNet encoder-decoder (over 120 FPS on its GPU). Self-supervised lightweight models such as Lite-Mono [159] and SC-DepthV3 [126] learn from video without labels. FastDepth and SC-DepthV3 are the two depth models Hailo ships in its Model Zoo for the Hailo-8L, and both appear in our benchmark. AsyncMDE [83] amortises a foundation model over time: a 97.5 M frozen slow path writes spatial memory, and a 3.8 M fast path runs at 161 FPS on a Jetson AGX Orin. Closest in spirit to our platform, Jalalnezhad et al. [46] run a lightweight monocular depth network at 14–20 Hz on embedded CPUs (Raspberry Pi 4 class) with behaviour-based control. Our work differs in three ways. The accelerator is an integer-only NPU rather than a GPU or a CPU. The network is a student of a foundation model rather

than a model trained from scratch. And the depth feeds a full mapping, planning and SLAM stack rather than a reactive behaviour.

2.3 Quantising transformers

Integer inference [45] is what makes NPUs efficient. Vision transformers are known to be hard to quantise because post-softmax and post-LayerNorm activations have long-tailed, power-law distributions. FQ-ViT [74], PTQ4ViT [157] and RepQ-ViT [71] address this with log-domain softmax quantisers, twin uniform ranges and scale reparameterisation, each of which needs support in the target hardware's quantiser. On the Hailo-8L, attention is executed as int8 with a uniform quantiser that we cannot change. Chapter 3 measures what that does to Depth Anything V2, and explains why, for this chip, removing attention through knowledge distillation [39] is more practical than better quantisation.

2.4 Navigating with predicted depth

MonoNav [120] fuses pre-trained transformer depth into a metric reconstruction and plans through it, flying a 37 g Crazyflie at 0.5 m/s, with depth computed off-board. MonoMPC [119] uses estimated depth not for direct collision checking but as context for a learned collision model inside a risk-aware MPC. MonoSpheres [97] explores large outdoor environments from a sparse monocular SLAM front end. Finke et al. [25] is the closest recent work. They fine-tune Depth Anything V2 for four onboard cameras, train a reinforcement-learning student policy in Isaac Lab, and run everything on a Jetson AGX Orin at 10 Hz, outperforming a 2D LiDAR on overhangs and low objects. Learned navigation policies such as ViNT [118] and NoMaD [122], and imperative planners such as iPlanner [150] and ViPlanner [114], map images or depth directly to paths. We deliberately keep the planning classical—hybrid A* [21], pure pursuit [17], and a reactive arc scorer in the tradition of the vector field histogram [10]—so that every failure can be attributed to perception, planning or localisation.

2.5 SLAM

ORB-SLAM3 [11] is the reference feature-based system. DROID-SLAM [129], DPVO [130] and MASt3R-SLAM [96] (15 FPS with GPU kernels) are dense or learned alternatives that need a GPU. On a Pi CPU shared with planning we use a deliberately small front end: ground-plane visual odometry that lifts tracked floor features [82] to metric points with the known camera height, optionally fused with wheel odometry and a gyro, plus a rotation-compensated image-stillness test that detects wheel slip (Chapter 6).

2.6 Simulation and hardware in the loop

We evaluate in NVIDIA Isaac Sim with Isaac Lab [98]. The yard is a Gaussian-splat reconstruction [55] of a real yard, rendered through NVIDIA's neural-reconstruction path, which builds on the 3DGUT formulation [145]. The robot computer is not simulated. The real

Raspberry Pi 5 and Hailo-8L receive every camera frame over the network and send back commands, which land after the measured round trip.

2.7 The state of the art in October 2026

This section surveys the literature as it stands at submission, family by family, and states for each what it means for our claims. Three developments since 2024 matter most. Depth models now absorb the camera and, increasingly, multiple views. Compact depth networks are now built by distilling those models, as ours is. And foundation models have entered SLAM, so far only on GPU-class hardware. The section ends with what remains ours once this is accounted for (Section 2.7.9).

2.7.1 Depth foundation models now carry the camera

Beyond the relative models of Chapter 2, metric depth has become a question of camera modelling. ZoeDepth added metric bins to a relative backbone [7]. UniDepthV2 [103] and UniK3D [102] predict the camera themselves; UniK3D replaces the pinhole with a spherical ray representation because pinhole metric models fail on wide lenses. MoGe [137] moved the target from depth to an affine-invariant pointmap that also recovers the field of view. MoGe-2 [138] made it metric and attributes its gains to refining real training data with synthetic labels, and MoGe-3 [60] refines the pointmap in a sparse 3D volume so that features no longer mix across depth discontinuities. Prompt Depth Anything [72] takes the opposite route and prompts the network with a cheap metric sensor. Diffusion models were reduced to a single step [35] but remain Stable-Diffusion sized.

In parallel, multi-view feed-forward reconstruction has absorbed the monocular task. DUSt3R [139] and MASt3R [66] regress pointmaps for image pairs. VGGT [135] predicts cameras, depth and pointmaps for up to hundreds of views in one pass, CUT3R [136] makes this recurrent for streams, π^3 [141] removes the reference view, and MapAnything [54] accepts optional intrinsics, poses or depth. Depth Anything 3 [73], now published at ICLR 2026, reports beating Depth Anything V2 at monocular depth and VGGT at pose with a plain transformer.

For us: every model in this family is a ViT with attention and LayerNorm throughout, and the metric ones need their camera model to be right. Ground-contact ranging (Chapter 5) needs only the per-column ordering of floor pixels plus one number, the camera height, so a relative student is sufficient. We use these models only as GPU references.

2.7.2 Small depth: everyone now distils, few report int8

The lightweight line used to train from scratch on one domain, supervised [144] or self-supervised from video [30, 104, 126, 159]. Raw speed on Raspberry-Pi-class hardware is not new. The Mobile AI challenges timed every entry on a Raspberry Pi 4 CPU, and the 2022 winners produced VGA depth at up to 27 FPS [42, 43], the rate our student reaches on the Hailo-8L. Their models were trained and tested on a single ZED-stereo dataset, however. In 2026 the field converged on the recipe of Chapter 3: ZipDepth [131] distils a foundation model into a 6.1 M-parameter reparameterisable encoder-decoder, DepthART [149]

builds a tiny model family with camera-conditioned metric fine-tuning, and AsyncMDE [83] amortises a frozen foundation model over time. All three report FP32 or FP16 throughput on GPUs (Jetson Nano to A6000). None reports an int8-only accelerator. Hailo now ships Depth Anything V1/V2 ViT-S at 224×224 for our chip, at 32.4 FPS for V2, and reports AbsRel 0.15 in float and 0.16 on hardware [33]. Chapter 4 finds that int8 attention erases obstacles in that same build. An image-averaged AbsRel need not reveal this, because obstacles occupy few pixels. Compactness does not imply quantisability either: in a preliminary experiment outside the benchmark, we compiled ZipDepth for a Qualcomm HTP and its int8 output collapsed structurally, with no improvement at 16-bit activations.

For us: the claim is not "fast small depth", which the MAI challenges already had. It is zero-shot, foundation-quality depth that survives an integer-only NPU with a fixed quantiser, verified on the chip itself.

2.7.3 Stereo as the reference, not the competitor

Our real-car reference is a RealSense D435. Its D4 ASIC computes a census cost [158] with semi-global aggregation [40, 56]. Its error grows with the square of range, $|\Delta z| = z^2|\Delta d|/(fB)$ with $|\Delta d| \approx 0.1\,\text{px}$ for a good passive pair [56]. The datasheet bounds Z-accuracy and RMS error at 2 % up to 2 m [110], and independent studies fit quadratic error models [34, 116]. Sunlight helps the D4xx unless glare or the sun enters the view [111]. Learned stereo followed monocular depth: RAFT-Stereo, CREStereo and IGEV [69, 75, 146, 147] set accuracy, and FoundationStereo [142], DEFOM-Stereo [48] and MonSter [13] added monocular-depth priors for zero-shot robustness. Fast-FoundationStereo [143] and Lite Any Stereo [49, 50] then brought that robustness to real-time budgets. We therefore trust no single matcher. A reference point requires the D4 ASIC, OpenCV SGBM and Fast-FoundationStereo to agree on the same IR pair, within 0.3–3 m, and each matcher is scored leave-one-out against the other two.

2.7.4 2D SLAM: a textbook algorithm with an unusual sensor

Each part of our SLAM has a well-known ancestor. Occupancy grids date to Moravec and Elfes [94]. Matching a scan against the robot's own grid with an exhaustive correlative window is Olson's real-time correlative scan matching [99]. Hector SLAM [58] and Cartographer [38] solve the same scan-to-map problem by Gauss–Newton and branch-and-bound. Keyframes, odometry edges, loop edges from a wide search, and a global re-solve form the pose-graph pattern [31, 81] of Karto [61], Cartographer and slam_toolbox [84], solved elsewhere with g2o [63] or iSAM2 [19, 51]. Before accepting a loop we check that the match peak is unique, a minimal guard against perceptual aliasing. Our Cauchy loss is the simplest robust kernel. It bounds the pull of one false closure. Switchable constraints [127], max-mixtures [100], DCS [1], RRR [65], PCM [89] and GNC [151] give stronger guarantees when many closures are wrong. tinySLAM [123] and its port BreezySLAM [68] show that grid SLAM fits a very small computer, and GMapping [32] is the particle-filter alternative.

For us: the new element is the sensor, not the algorithm. A pseudo-scan from monocular depth has *per-column systematic* error that grows with range, unlike the near-independent beam noise these systems were tuned for. Cartographer and slam_toolbox can consume our scan unchanged, which makes them the natural baselines.

2.7.5 Visual and learned SLAM

Feature-based, direct and semi-direct visual SLAM [11, 23, 24, 26, 27, 95] run on a CPU but recover monocular geometry only up to scale. VIO [106] restores scale with an IMU, RTAB-Map [64] with depth. We take scale from the camera height instead, as monocular driving systems do from the road plane [121]. Dense RGB-D pipelines such as Open3D's [14, 162] use the depth directly. Learned SLAM moved from learned optimisation (DROID-SLAM, DPVO, DPV-SLAM [76, 129, 130]) to foundation-model front ends: MASt3R-SLAM [96], SLAM3R [78], MegaSaM [70], VGGT-SLAM [87], VGGT-Long [20], AIM-SLAM [47] and VGGT-SLAM++ [88]. VGGT-SLAM 2.0 [86] verifies loop closures with a VGGT attention layer and runs online on a ground robot, on a Jetson Thor. ScaRF-SLAM [160] argues for the split we adopted at a far smaller scale: classical tracking, because errors in a geometric foundation model would otherwise propagate into the pose, and the learned model only for mapping. We found no published measurement of these systems on a Raspberry Pi 5. Run offline on a GPU over our recorded real-world drive, the classical and learned systems above are accuracy references. None of them is a deployment alternative on our platform.

2.7.6 Monocular depth as a range sensor

Our sensor model has direct ancestors, and they narrow what we can claim. Ulrich and Nourbakhsh [133] already described ranging from the lowest obstacle pixel in each image column under a flat-floor, no-overhang assumption, as "a common approach". Michels, Saxena and Ng [92] learned a per-column distance from one image and drove a small RC car with it. The Stixel World [4] made column-wise free space the standard intermediate representation for stereo, and StixelNet [67] predicted it from one image. Turning depth into the input a downstream system expects is the Pseudo-LiDAR argument [140, 156], and ROS has long shipped `depthimage_to_laserscan` [113]. Tightly coupled depth priors inside VO [79, 128, 154, 155] are the alternative to our loosely coupled range measurement. Closest to us, Barhoum and Pyrkin [5] fed a monocular "virtual scanner" into Cartographer, in simulation, with scale fitted against an RGB-D camera. Marsal et al. [90] rescale Depth Anything V2 with VINS-Mono points into an off-road elevation map. Pushparaju et al. [105] calibrate a distilled student from the valid pixels of a failing ToF camera. A 2026 Aalborg thesis on our exact platform, a Raspberry Pi 5 with a Hailo accelerator, concludes that relative depth is best used to *select* a target and a calibrated ground-plane homography to range it [115]. That is the same division of labour as our ground-contact scan, though their work studies distance estimation, not navigation.

2.7.7 Long-term operation

Appearance-based localisation [3, 6, 18, 28, 44, 53, 93] handles lighting change either by remembering several experiences [8, 15] or by mapping images into an invariant space [91]. Benchmarks show day–night and seasonal change to be the dominant failure [80, 85, 117]. Our map stores geometry and matches scans, so lighting can reach it only through the depth network. The relevant literature is therefore depth robustness: RoboDepth [59], Depth Anything at Any Condition [125] and weather-conditioned variants [148]. We have not measured relocalisation across sessions.

2.7.8 How our evaluation follows the literature

ATE and RPE follow the TUM benchmark [124]. Alignment may absorb only the degrees of freedom an estimator cannot observe [161]. A system that claims metric scale, as ours does through the camera height, must therefore be scored after a rigid alignment. Monocular baselines are up to scale and can only be scored after a Sim(3) alignment [134], which hands them the scale for free. The two ATEs are different quantities and must be reported side by side, never ranked in one column. KITTI's segment metric [29] assumes runs far longer than ours, and relation-based errors [62] remain an option where no global reference exists. Depth is scored against sparse reference points, as in the KITTI depth benchmark [132]. Relative models are aligned on points other than the scored ones, plus a single per-sequence scale, because per-frame alignment on the scored pixels flatters them. Boundary metrics [57] would target obstacle edges more directly than AbsRel. Every hardware-in-the-loop number uses at least five seeds with interval estimates, following the statistical-precipice critique [2, 16]. Real-world comparisons use a frame-cluster bootstrap, because consecutive frames are not independent samples.

2.7.9 What remains ours

Table 2.1 updates Table 2.2 with the closest 2024–26 work. The literature leaves the following claims, which are narrower than "first monocular SLAM on a Raspberry Pi".

1. *Not new:* ground-contact ranging [92, 133], monocular pseudo-scans for 2D SLAM [5], Pi-class frame rates [43], distilling foundation depth into small networks [131, 149], correlative matching and robust pose graphs [38, 99].
2. *Ours:* the combination, measured. Foundation-quality depth survives an int8-only NPU at 27 FPS, verified on the chip, where both vendor and concurrent students are untested or fail. A pseudo-scan takes its metric scale from the camera height alone, with no RGB-D, LiDAR or SLAM points to calibrate it, and gets three failure-analysis fixes. Mapping, planning and SLAM run on the same Raspberry Pi 5. The whole loop is evaluated in hardware-in-the-loop with the real board (at least five seeds per configuration) and on a real car against a stereo-consensus reference.
3. *Open:* cross-session and lighting robustness, and a stronger robust back end. A head-to-head against slam_toolbox or Cartographer fed with the same scan would show whether our SLAM, rather than our sensor, limits accuracy.

2.8 Positioning

Table 2.2 contrasts the closest systems. To our knowledge this is the first work that (i) runs a foundation-model-quality depth network on a 13-TOPS, int8-only Raspberry Pi accelerator, (ii) builds metric mapping, global planning and SLAM on it within the Pi's own budget, and (iii) evaluates the whole loop with the real board in a reconstructed real environment, with an explicit decomposition of the remaining error.

Table 2.1. The closest 2024–26 work, as of October 2026.

	Depth and where it runs	Metric scale from	Mapping / evaluation
Barhoum & Pyrkin [5]	relative CNN, desktop	regression to an RGB-D camera	Cartographer; ROS 2 simulation, no numbers
Ruás Branco [115]	embedded relative depth, Raspberry Pi 5 + Hailo	ground-plane homography	distance estimation; no SLAM
Marsal et al. [90]	Depth Anything V2, GPU	VINS-Mono points	2.5D elevation map; sim + real off-road
Pushparaju et al. [105]	distilled student, Jetson Orin Nano, 218 FPS	valid ToF pixels	costmap fusion; closed-loop sim
ZipDepth / DepthART [131, 149]	distilled students, FP32/FP16 GPU	— / camera-conditioned	depth only
VGGT-SLAM 2.0 [86]	VGGT, Jetson Thor	feed-forward reconstruction	dense SLAM; real ground robot
This thesis	**attention-free student, Hailo-8L int8, 27 FPS**	**camera height only**	**2D SLAM + loop closure on the Pi; HITL + real car vs stereo consensus**

Table 2.2. Closest systems that navigate with predicted monocular depth.

	Depth model and where it runs	Navigation	Evaluation
MonoNav [120]	pre-trained transformer, *off-board*	metric fusion + motion primitives	real indoor MAV flights
Finke et al. [25]	fine-tuned Depth Anything V2, 4 cameras, Jetson AGX Orin, 10 Hz	RL policy (teacher–student)	Isaac Lab + real robot
MonoMPC [119]	estimated depth as context	learned collision model + MPC	sim + real
Jalalnezhad et al. [46]	lightweight CNN, embedded CPU, 14–20 Hz	behaviour-based control	real indoor/outdoor
This thesis	**distilled attention-free student, Hailo-8L int8, 27 FPS**	**ground-contact scan, hybrid A*, SLAM, all on the Pi**	**real-time HITL with the real board, $\geq$5 seeds, error budget**

Chapter 3

An attention-free depth student for the Hailo-8L

3.1 Platform

The robot computer is a Raspberry Pi 5 (four Cortex-A76 cores) with the Raspberry Pi AI Kit: a Hailo-8L module on an M.2 HAT, rated at 13 TOPS and about 1.5 W typical [109] (the module has no power sensor that HailoRT can read, so this thesis reports no measured power). Networks are compiled offline with Hailo's Dataflow Compiler into a HEF file and executed through HailoRT. The Hailo-8L is an integer accelerator: on this part every layer runs with 8-bit activations and weights, including the matrix products and softmax of attention.

3.2 Why transformer depth breaks on this chip

Our first step was the obvious one: export Depth Anything V2-Small [153] at 224×392 (a 16×28 grid of 14-pixel patches plus a class token, 449 tokens), calibrate it on domain images, and compile. On real robot video the result looked right (median Pearson 0.99 against the float model). On the yard renders it did not: on a third of held-out frames it *erased* obstacles, producing depth maps in which a crate or a person dissolves into the floor (Figure 4.3, fourth column).

We traced this to attention in int8 (Figure 3.1). A transformer head whose attention is spread over many tokens produces softmax weights of order $1/449 \approx 0.0022$. A uniform 8-bit quantiser over $[0, 1]$ has a step of $1/255 \approx 0.0039$. A diffuse head is therefore rounded to zero or to one step, and what it was averaging is lost. This is exactly the long-tail problem that ViT quantisation methods attack with log-domain or twin-range softmax quantisers [71, 74, 157]. The Hailo-8L toolchain applies its own int8 scheme to these layers (a8_w8_a8), and we found no supported way to give softmax a different quantiser. We tried the remedies that were available:

- re-calibration on domain images plus the compiler's quantisation fine-tuning moved the share of held-out frames with Pearson < 0.8 only from 48 % to 37 %;
- sensor-noise and sharpening camera models in calibration left 30–39 % of frames broken;

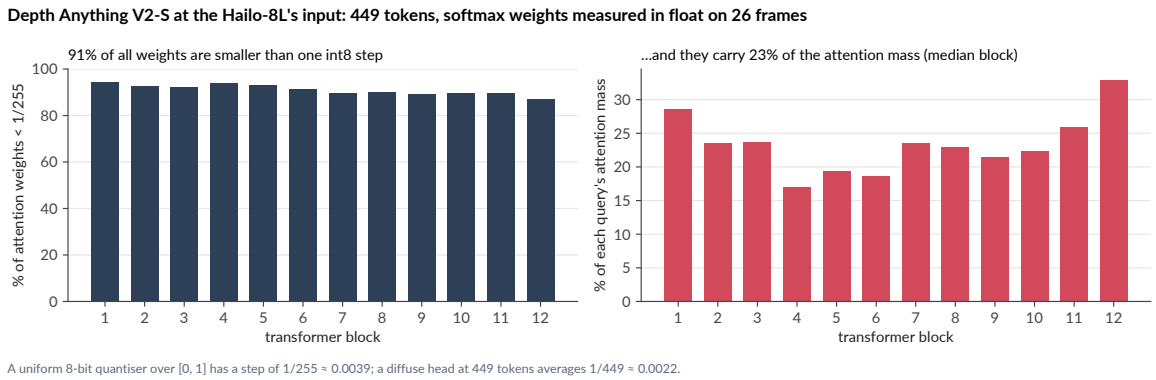


Figure 3.1. Measured in float Depth Anything V2-Small at the Hailo-8L's 224×392 input (449 tokens) on 26 held-out sim and real frames: 91 % of all softmax weights are smaller than one step of a uniform 8-bit quantiser over $[0, 1]$, and they carry 17–33 % of each query's attention mass (median 23 %; most in the last block, which feeds the depth head).

- Hailo's own Model Zoo build of Depth Anything V2-Small (224×224) shows the same blockiness and object loss (Figure 4.3, fifth column; Chapter 4).

We measured how much of the computation lives below that step (Figure 3.1). Across the twelve blocks, 91 % of the weights are below one int8 step, and they carry 17–33 % of every query's attention mass. That is the mass an 8-bit uniform quantiser rounds to zero or to a single step.

The failure is structural: int8 cannot represent diffuse attention at a few hundred tokens. Better calibration does not fix it; removing attention does.

3.3 The student

We therefore distil Depth Anything V2-Large (335 M parameters) into a network with *no attention at all* (Figure 3.2). A ResNet-34 encoder [36] (ImageNet initialisation) feeds a U-Net decoder [112] built only from convolution, batch normalisation, ReLU, 2× bilinear upsampling and skip concatenation, all of which quantise well. It has 24.5 M parameters, almost exactly Depth Anything V2-Small's 24.8 M, so the comparison isolates architecture from capacity. Input is RGB at 288×512, the aspect of the robot camera. Output is relative inverse depth (disparity) at full input resolution.

Loss. The student is trained with the scale- and shift-invariant L1 loss of MiDaS [108] plus a four-scale gradient-matching term. Both prediction and target are normalised per image by median and mean absolute deviation, so only the *shape* of the depth map is supervised. This is the right target for a relative teacher and, as Chapter 5 shows, all the navigation stack needs.

Data. The teacher labels three sources:

- 7,230 simulated point-of-view frames from hardware-in-the-loop runs of scenes 00–06 (every 0.2 s);

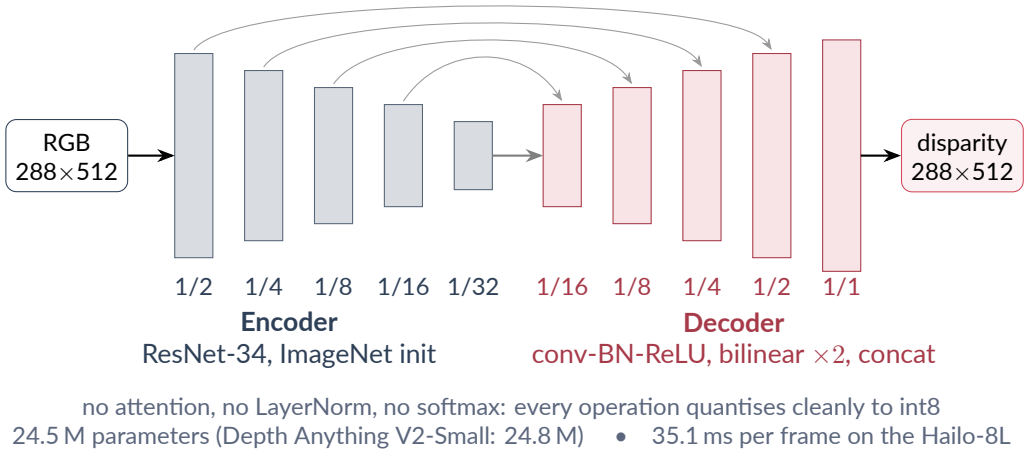


Figure 3.2. The attention-free student. Grey arrows are U-Net skip connections.

- 720 real frames from the robot camera and calibration sets;
- later, 2,958 random-layout renders and 1,304 frames from 36 local videos.

Every frame of scenes 07–09 (crates, people, barrels) and the real robot video are held out completely.

Stages.

v1 distilled from Depth Anything V2-Large on the first set. Held-out Pearson to the teacher: 0.990 (sim) and 0.998 (real video).

v2 the same with the extra data, 20 epochs. It did *not* improve the novel-object scene (crates: frames below $r = 0.8$ rose from 14.5 % to 21.0 %). Closing a novel-object gap needs a far larger and more varied corpus.

v3 v1 fine-tuned for 8 epochs on *rendered* ground-truth disparity for the simulated frames, with the teacher kept for real frames (13,902 simulated + 2,024 real training frames, AdamW, one-cycle learning rate 3×10^{-4}, batch 16, about 150 s per epoch on an RTX 5090). The teacher's own relative depth had made v1 range simulated props about 30 % short; v3 brings that bias to about -7 %.

v5 v3 with the loss weighted $\times 4$ on the image rows below the horizon (floor and object bottoms). It reduced spurious near-floor hits by a quarter but did not resolve obstacle bases better (Section 5.5). In the loop it is not resolved from v3 with simulator pose (7.9 vs 7.9, ten seeds) and is ahead, though not significantly, on the robot's own pose (Chapter 7). It is the deployed student.

3.4 Compilation and on-chip fidelity

The student compiles with normalisation folded into the HEF and optimisation level 1, calibrated on domain frames. One finding matters for anyone repeating this: the compiler's own emulator check reported a correlation of -0.41 between int8 and native for this graph,

a red flag that turned out to be false. Replaying the HEF on the real chip against the float model gives:

Student v3, Hailo-8L int8 vs the same weights in float, 500 frames (400 held-out sim + 100 real): Pearson median 0.99995, 5th percentile 0.99897. The quantised network is, for every practical purpose, the float network. Judge an NPU build by replay on the chip, not by the compiler's estimate.

The deployed v5, compiled with the same recipe, replays at Pearson 0.99993 (5th percentile 0.99569) on the same frames.

On the Pi the student takes 35.1 ms on the NPU, or 37.0 ms per frame including pre- and post-processing: 27.1 FPS at batch 1 and 59 FPS at batch 8. Our int8 Depth Anything V2-Small build takes 57.7 ms (16.6 FPS) for fewer pixels. The student therefore delivers 1.7× the pixels at 1.6× the speed, with no visible quantisation loss.

Chapter 4

Benchmark: depth on the edge

4.1 Protocol

Frames. 400 simulated point-of-view frames from 36 hardware-in-the-loop episodes of the three held-out scenes (07 crates, 08 people and cones, 09 barrels), each with the renderer's metric depth, plus the 100 held-out frames of real robot video, which have no ground truth.

Models. Every depth network we could run on the Hailo-8L: our students (v5, deployed; v3 and v1 for reference), our int8 Depth Anything V2-Small build, and Hailo's Model Zoo builds of Depth Anything V2-Small (224×224), SC-DepthV3 [126] and FastDepth [144]. All of them run *on the chip*, with the outputs read back. As float references on the GPU we use Depth Anything V2 S/B/L [153] and four Depth Anything 3 variants [73]. These cannot run on the Pi at a useful rate.

Alignment and metrics. Each model sees a centre crop at its own input aspect, and ground truth is cropped to match. Predictions are converted to disparity. Following the standard relative-depth protocol [108], a per-frame least-squares scale and shift aligns them to ground-truth disparity on valid pixels (0.1–20 m). We report AbsRel and δ_1 (fraction of pixels with $\max(\hat{z}/z, z/\hat{z}) < 1.25$) on two regions. *All* is every valid pixel. *Obstacles* is the set of pixels within 3 m that, back-projected with the car's ground-truth pose, lie on a prop's collider. The obstacle region matters because the renderer's depth buffer has speckle and floaters near the floor (visible in Figure 4.3); restricting to pixels on known props removes that noise and measures what the planner actually reads. On real video the metric is the Pearson correlation with the Depth Anything V2-Large teacher in each model's own crop.

Speed. Frames per second on the Raspberry Pi 5 + Hailo-8L, batch 1, sequential: CPU pre-processing, NPU, read-back and post-processing, median over 500 frames. For reference, the Pi's four Cortex-A76 cores alone (onnxruntime, four threads) take 455 ms per frame for the student (2.2 FPS) and 381 ms for Depth Anything V2-Small at 224×392: the Hailo-8L makes the student 12× faster. On the RTX 5090 laptop GPU (idle, batch 1, including each model's own pre- and post-processing) the float models take 38.8 ms (Depth Anything V2-S), 69.9 ms (Depth Anything V2-B), 175.3 ms (Depth Anything V2-L) and about 40 ms (Depth Anything 3 variants); the student takes 4.0 ms.

Table 4.1. Depth on held-out frames. AbsRel/δ_1 on 400 simulated frames with metric ground truth; "Real r" is the median Pearson to Depth Anything V2-Large on 100 real robot frames; FPS measured on the Raspberry Pi 5 + Hailo-8L (batch 1). Bold: best on the chip.

Model	Runs on	Input	Params (M)	AbsRel ↓		δ_1 ↑		Real r	FPS
				all	obstacles	all	obstacles	vs DAv2-L	Pi+Hailo
Ours: distilled student v3	Hailo-8L int8	288×512	24.5	**0.269**	0.204	0.674	0.776	0.995	27.1
Ours: student v5 (floor-weighted)	Hailo-8L int8	288×512	24.5	0.269	0.197	0.669	0.762	0.996	27.1
Ours: student v1 (no sim-GT fine-tune)	Hailo-8L int8	288×512	24.5	0.320	**0.188**	0.613	0.746	0.996	27.1
DAv2-S, our int8 build	Hailo-8L int8	224×392	24.8	0.477	0.330	0.423	0.591	0.982	16.6
DAv2-S, Hailo Model Zoo	Hailo-8L int8	224×224	24.8	0.354	0.334	0.593	0.619	0.979	21.6
SC-DepthV3, Hailo Model Zoo	Hailo-8L int8	256×320	–	0.414	0.352	0.506	0.688	0.950	53.6
FastDepth, Hailo Model Zoo	Hailo-8L int8	224×224	–	0.530	0.767	0.344	0.477	0.709	121.3
Ours: student v3 (float twin)	GPU fp32	288×512	24.5	0.266	0.200	0.679	0.780	0.995	–
Ours: student v5 (floor-weighted)	GPU fp32	288×512	24.5	0.270	0.200	0.670	0.758	0.996	–
Depth Anything V2-S	GPU fp32	518 short side	24.8	0.283	0.150	0.651	0.813	0.999	–
Depth Anything V2-B	GPU fp32	518 short side	97.5	0.289	0.151	0.637	0.800	0.999	–
Depth Anything V2-L (teacher)	GPU fp32	518 short side	335.3	0.284	0.144	0.643	0.805	1.000	–
Depth Anything 3-Small	GPU fp32	native	34.3	0.315	0.172	0.611	0.760	0.988	–
Depth Anything 3-Base	GPU fp32	native	135.4	0.304	0.188	0.616	0.735	0.985	–
Depth Anything 3-Mono-Large	GPU fp32	native	334.2	0.295	0.171	0.568	0.747	0.983	–
Depth Anything 3-Metric-Large	GPU fp32	native	334.2	0.374	0.239	0.574	0.687	0.988	–

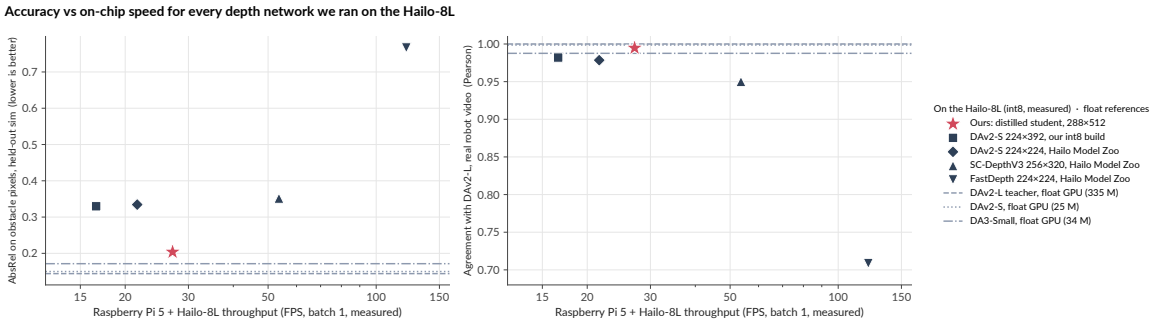


Figure 4.1. Accuracy against measured on-chip throughput. Left: AbsRel on obstacle pixels (lower is better). Right: agreement with Depth Anything V2-Large on real robot video. Horizontal lines are float GPU models that cannot run on the Pi at this rate.

4.2 Results

Table 4.1 and Figure 4.1 summarise the benchmark.

1. **On the chip, the student is in a different class.** Overall AbsRel 0.269 against 0.354 (Hailo's Depth Anything V2-S), 0.414 (SC-DepthV3), 0.477 (our Depth Anything V2-S build) and 0.530 (FastDepth); δ_1 0.674 against at most 0.593. On obstacle pixels the gap is just as clear: 0.204 against 0.330–0.767. On real video it tracks the teacher at $r = 0.995$, where Hailo's Depth Anything V2-S reaches 0.979, SC-DepthV3 0.950 and FastDepth 0.709.

2. **Speed is not traded for this.** At 27.1 FPS the student is faster than both Depth Anything V2-S builds on the same chip (21.6 and 16.6 FPS) while processing 2.9× and 1.7× more pixels. Only the two small self-supervised zoo models are faster (54 and 121 FPS), at a large accuracy cost.

3. **Against float foundation models on a GPU, the student gives up a few points on obstacles but not overall.** Depth Anything V2-L, the teacher, reaches 0.144 AbsRel on ob-

stacles against the student's 0.204. Overall the student is *better* (0.266 float, 0.269 on the chip, against 0.283–0.289 for Depth Anything V2 S/B/L), because v3 was fine-tuned on rendered ground truth of this domain. Depth Anything 3 is not better than Depth Anything V2 here (0.17 obstacle AbsRel for the Small and Mono-Large variants).

4. **v1 vs v3.** Fine-tuning on rendered ground truth improves the global metrics (all-pixel AbsRel 0.320→0.269) and the metric range bias, but slightly worsens obstacle pixels (0.188→0.204). In the closed loop the difference is not resolved (7.2 vs 7.2) (Chapter 7). We report the trade rather than pick a favourable metric. v5, the deployed student, matches v3 overall on the chip (0.269) and recovers part of the obstacle loss (0.197).

4.3 What the benchmark does and does not show

The simulated frames come from a reconstruction of the same yard the student was fine-tuned in, although from held-out scenes with objects placed differently and, for crates, never seen in training. The real frames are a different camera in a different building. The student's agreement with its teacher there ($r = 0.995$) shows that sim fine-tuning did not damage real-world behaviour, but agreement with a teacher is not accuracy against ground truth, which we do not have for real frames. Finally, the obstacle region measures depth on props. It does not measure the thin, low structures (cone base plates, chair tubes, feet) that turn out to matter most in the loop; Section 5.5 measures those directly.

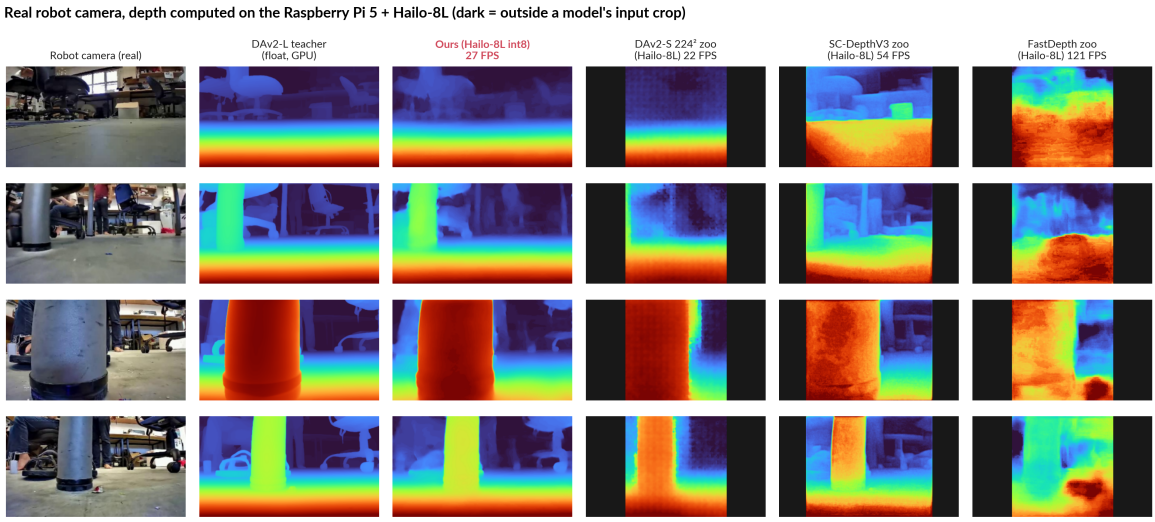


Figure 4.2. Real robot camera. Each column is the depth *as computed on the Hailo-8L* (except the GPU teacher). Dark areas lie outside a model's input crop: the 224×224 Model Zoo builds see only a centre square of the wide camera.

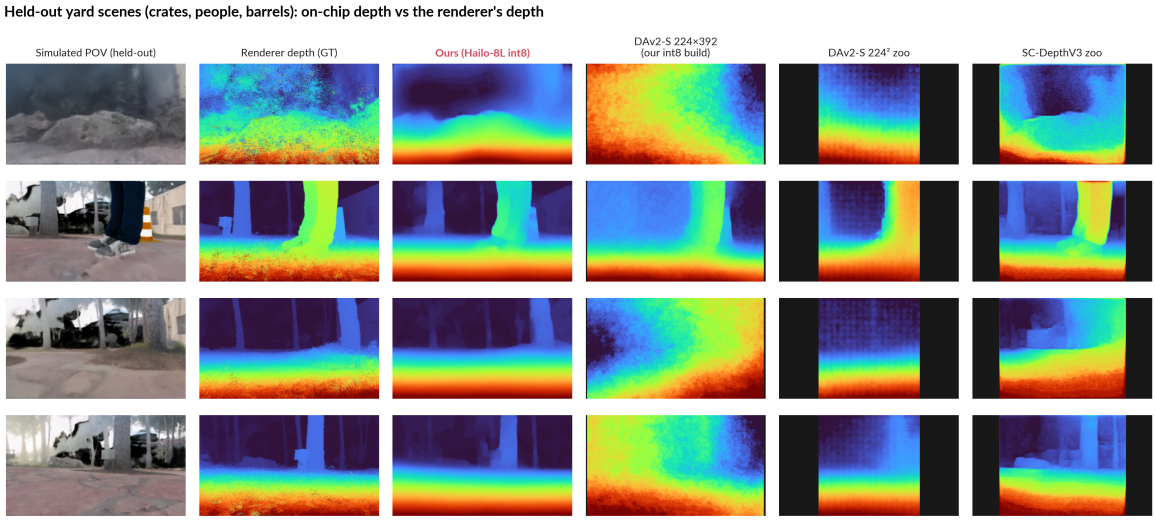


Figure 4.3. Held-out simulated scenes with the renderer's depth (note its speckle near the floor). Our int8 Depth Anything V2-S build scrambles whole frames (int8 attention, Section 3.2); the student stays close to the ground truth.

Chapter 5

From relative depth to metric space

The student predicts disparity up to an unknown scale and shift, $d = a/Z + b$, and its scale drifts from frame to frame. A planner needs metres. Rather than learn metric depth (the route of Metric3D, UniDepth and Depth Pro [9, 41, 101]), we take the metric from geometry the robot already knows: its camera height.

5.1 Ground-contact geometry

For a level camera at height h with focal length f_y and principal row c_y, a floor point imaged at row $v > c_y$ lies at depth

$$Z_{\text{floor}}(v) = \frac{f_y\, h}{v - c_y}. \tag{5.1}$$

An obstacle standing on the floor meets it at some row v_c; its range is $Z_{\text{floor}}(v_c)$, whatever the network's scale. The depth network's only job is to say *where in each image column the floor ends*, a question about the *shape* of the depth map, which is exactly what a scale- and shift-invariant loss supervises. The image's columns then become a 69-degree, 4-metre "laser scan" with one range per column (every second column at 288×512).

5.2 The scan

Floor profile. For each row, the 25th percentile of disparity over the central 80 % of columns models the floor (obstacles covering less than about two thirds of a row sit above it). The profile is made monotone (the floor only gets nearer going down the image), limited to rise at most 1.3× between frames, and smoothed in time. The far-field offset b is the 3rd percentile of disparity above the horizon.

Ratio detector. For a pixel at row v, $(d - b)/(f(v) - b) = Z_{\text{floor}}/Z$ is independent of the network's scale. A pixel standing at least $H_{\min} = 5$ cm above the floor at its range has ratio $\geq 1/(1 - H_{\min}/h)$. Scanning each column from the bottom, the first run of five flagged rows is the obstacle's contact, and its range follows from the contact row.

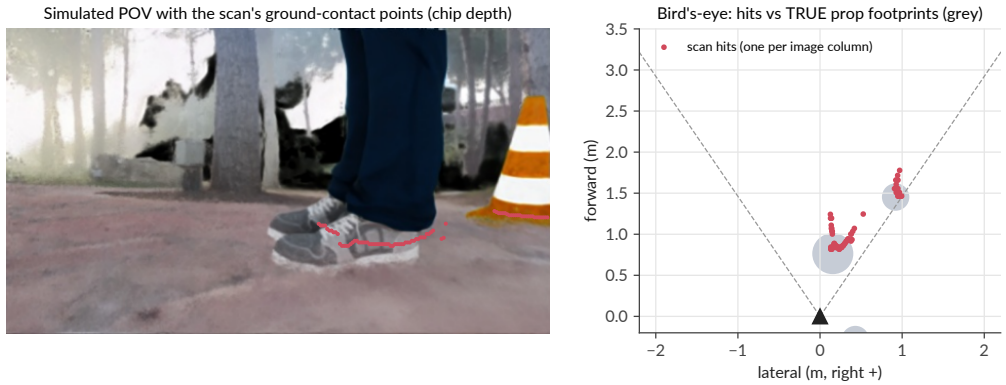


Figure 5.1. The scan on a held-out frame, from depth computed on the Hailo-8L. Left: one ground-contact point per image column. Right: the same hits in bird's-eye view against the *true* prop footprints (the person's collider is a conservative 0.25 m cylinder around the feet).

Vertical-face detector. On the floor, disparity grows linearly down a column, so $s = \frac{\partial d}{\partial v}\frac{v-c_y}{d-b} \approx 1$. On a vertical face disparity is constant, so $s \approx 0$. A run of 16 rows with $s < 0.3$ marks a face. This detector does not use the floor profile, so it still sees a box that fills the whole lower image (which the percentile floor absorbs). Measured slopes are 0.01–0.13 on a box at 0.45 m against 0.77–0.94 on clean floor.

Height. Following the contact disparity upward gives each obstacle's top height. Splat floaters and leaves are a few centimetres tall; legs, cones and boxes are not. Only hits at least 6 cm tall are mapped (except within 0.6 m, where blurred thin legs read short). On six recorded episodes this cut phantom hits from 2,376 to 75 while keeping 56 % of the hits on props.

Two fixes found by failure analysis.

- *Horizon-anchored floor.* The monotone floor profile was originally accumulated from the top of the image. Under a chair, the seat (near, *above* the horizon) was copied into every floor row, so the floor read nearer than the chair's own tubes. Even ground-truth depth then mapped a tube at 0.68 m as 1.16 m. Starting the accumulation at the horizon gives 0.76 m. Together with the planner fixes of Chapter 6, the chair scene went from 0/5 to 3/5 clean.
- *Floating overhangs.* A seat or table top 0.25–0.60 m up with open space under it is mapped as an obstacle at its own range, ranged with the floor profile as a ruler. In the chair and person scenes, 82–91 % of the columns this rule adds are confirmed by the same rule on ground-truth depth (59–76 % around crates and cones).

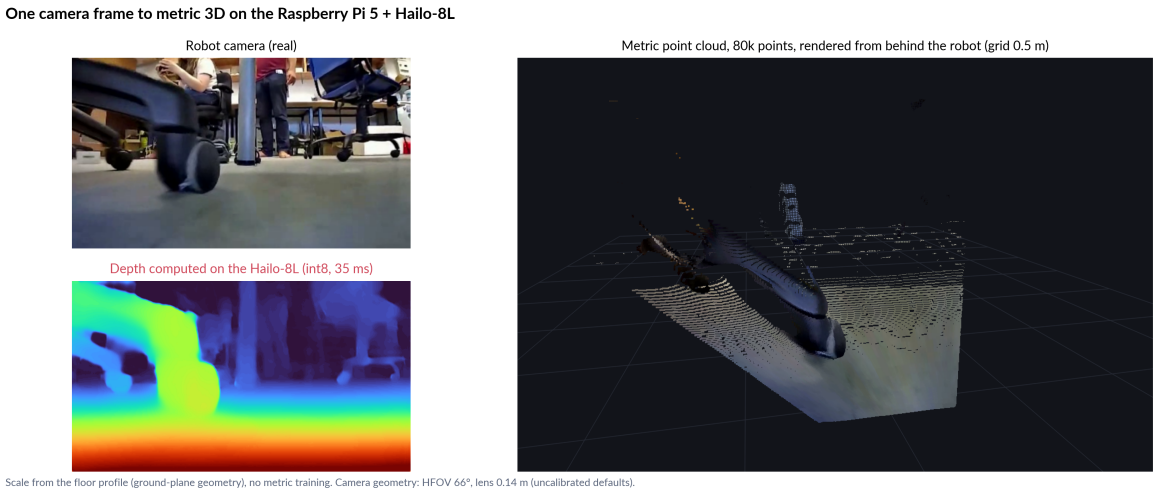


Figure 5.2. Real robot frame to metric 3D on the Pi: chip depth, scale from the floor profile, flying pixels removed. Camera geometry here uses uncalibrated defaults (66° horizontal field of view, lens 0.14 m).

5.3 Point clouds

The same floor profile is also a ruler for every pixel. Since $(f(v) - b)$ is proportional to $1/Z_{\text{floor}}(v)$, the product $K = Z_{\text{floor}}(v)\,(f(v) - b)$ is constant over floor rows, and any pixel's metric depth is $Z = K/(d - b)$. Back-projecting with the camera intrinsics gives a metric, coloured point cloud from one frame, with no metric training (Figure 5.2). We drop "flying pixels" at depth discontinuities (where a 3×3 neighbourhood spans more than 8 % in depth). We use the cloud for visualisation and overhang detection, not for ranging obstacles. Per pixel it is less robust than the contact scan: median error -2 to $+8$ % on obstacle pixels, but a long tail (p90 +44 to +167 % at object edges).

5.4 The map

Scan hits accumulate in a 5 cm grid of hit counts in the world frame. A cell becomes an obstacle after three hits *and* if it holds at least 30 % of the hits of the densest cell within 0.2 m. The absolute count suppresses single-frame noise: on real Hailo depth, one hit kept 53–102 phantom cells per episode and two hits 10–42; the deployed three is stricter still. The relative test stops an obstacle's sparse fringe from slowly growing it shut: a 0.9 m barrel gate otherwise read closed. Cells are never cleared. Clearing cells that later rays see through erased thin chair legs, which the Hailo's depth does not hit every frame. The world is static in our scenes. Rays also mark *seen* free space, which the planner uses (Chapter 6).

5.5 The measured limit: low bases

The remaining perception error is not in the depth model's scale but in thin, low structure. For each prop fully in view we measure how far the outermost scan hit lies *inside* the true collider edge, per side (Table 5.1).

Table 5.1. Scan side deficit (median, cm; positive = the scan sees the object narrower than its footprint) and raw near-hit phantom share, as a function of the height threshold $H_{\min}$, for ground-truth and student depth.

$H_{\min}$	GT depth, deficit	GT, phantoms	Student, deficit	Student, phantoms
5 cm (deployed)	+5.5	6.1 %	+5.1	13.7 %
3 cm	+1.7	11.5 %	+4.2	20.1 %
2 cm	−0.5	19.5 %	+2.5	24.7 %

Two facts follow. First, at the deployed threshold the ground-truth and student scans are *equally* narrow (+5.5 vs +5.1 cm). The gap is the threshold, not the network: cone base plates, sign and bin bases and shoes sit below 5 cm. Second, lowering the threshold closes the gap for ground-truth depth but not for the student, which blurs the floor-to-base boundary. Fine-tuning with the loss up-weighted below the horizon (v5) reduced phantoms (13.7→10.2 %) but not the deficit. A colour cue does close it for cones (cone deficit +5.7→+0.5 cm, no new phantoms). In the closed loop, however, the larger cones narrowed the slalom gaps and cost as much as they gained (Chapter 7).

Chapter 6

Onboard autonomy: planning, control and SLAM

Everything in this chapter runs on the Raspberry Pi 5 CPU while the Hailo-8L computes the next depth map.

6.1 Global planning: hybrid A* on sensed obstacles

Search. The planner is a hybrid A* [21] over continuous rear-axle poses (x, y, θ) with 0.1 m cells and 72 heading bins. Each expansion applies ten motion primitives, five curvatures forward and backward, each 0.25 m long. A path is collision-checked with a three-circle footprint (radius 0.16 m) against the sensed obstacles and against a *prior* drivability map of the yard derived from its reconstruction. Reversing costs twice as much as driving forward, and a change of direction costs an extra 0.6.

Heuristic. The cost-to-go is the larger of two admissible estimates: an obstacle-aware 2D geodesic distance to the goal, and a precomputed table of the obstacle-free *non-holonomic* cost of this exact lattice. The second matters: goals beside or behind the car need a reverse that holonomic heuristics cannot see. Without it, A* flooded 40,000 expansions on a recorded failure that needs one reverse.

Planning within the car's real turning ability. The simulated Traxxas turns a 0.95 m radius at full lock where the bicycle model says 0.86 m. Plans at full lock saturated the follower, and the car drifted to the outside of the turn, around the very obstacle it was curving past. Primitives are therefore planned at 75 % of lock (a 1.17 m radius), which leaves the follower steering margin.

A safety cascade. A first search uses a 0.10 m clearance margin (2 s budget). If it fails, a second relaxes it to 0.06 m and ignores the prior terrain map within 0.6 m of the start (1 s); the physics can let the car into ground the prior marks non-drivable, and it must be able to drive out. If the accepted path hugs its margin, a *wide* search at 0.20 m replaces it when at most 1.6× longer (1 s). Obstacle cells the car already overlaps when planning starts (a contact) are ignored within 0.3 m of the start, so the car can back out of them, and are hard obstacles beyond it. Chapter 7 shows why that last rule matters.

Unseen space. Space no ray has observed is passable but costs three times more within 0.9 m of the start. The camera sees $\pm 34^\circ$, and a hard turn swings the body into space beside the car that it never looked at.

Asynchronous execution. Searches run in a worker process; the car holds still while a plan is computed and otherwise follows the current one. On the Pi a search takes 110 ms at the median and 547 ms at the 90th percentile (993 recorded plans, Figure 6.1).

6.2 Control and safety behaviours

A pure-pursuit follower [17] (look-ahead 0.45 m, cut to 0.35 m and throttle 0.3 where the path passes within 0.15 m of an obstacle) tracks one direction segment at a time. Short three-point-turn legs are replayed open-loop because the look-ahead exceeds them. Re-planning is triggered when:

- a newly sensed obstacle blocks the path ahead;
- the car leaves the path by more than 0.25 m;
- a segment takes far longer than it should;
- the car is *wedged*: commanded throttle but no progress for 1.5 s. A "bump" obstacle is then placed at the bumper; this is how the car learns about things it cannot see.

When no plan exists the car backs out along its own recent trail, at most three times. A retreat that makes no progress is straightened, and a pinned straight retreat is treated as a wedge. The Traxxas ESC interprets reverse after forward as a brake. Every reverse therefore passes through brake (0.30 s) and neutral (0.15 s), and the simulator models the same gate. A reactive layer scores 21 constant-curvature arcs on clearance and geodesic progress [10], as a fallback when the global planner has nothing.

6.3 SLAM

Ground-plane visual odometry. About 400 features in the floor band (up to 1.5 m ahead) are tracked frame to frame with forward–backward Lucas–Kanade [82], lifted to metric floor points with the camera height (the same geometry as the obstacle scan), and a RANSAC 2D rigid fit gives $(\Delta x, \Delta z, \Delta\theta)$. The visual-odometry pose feeds a 2D log-odds occupancy map with scan matching.

Wheel odometry and a slip guard. The car can carry a rear-axle encoder and a gyro, and the SLAM step accepts $(\Delta s, \Delta\theta)$ directly. In simulation the encoder is the rear-wheel joint velocity times the wheel radius, and the gyro is the true yaw rate plus a $0.1^\circ/\text{s}$ bias and $0.3^\circ/\sqrt{\text{s}}$ noise. Wheels slip when the car pushes against something, so an encoder alone ran up to 5 m off. The guard declares the car *still* when the rotation-compensated median floor flow is below 2 px, and only when the encoder claims at least 2 cm per frame. Without that second condition, a slow legitimate reverse ($\approx$1 px of floor flow) was zeroed and reverse odometry read 7 % short. With it, the shortfall is 3 %.

Table 6.1. Self-localisation alternatives, offline on 50 recorded 35-second episodes (absolute trajectory error).

Odometry source	ATE median (m)	ATE p90 (m)
Ground-plane VO (camera only)	0.42	1.32
VO translation + gyro heading (an IMU alone)	0.36	1.04
Encoder + VO heading	0.22	1.18
Encoder + Ackermann heading from the steering command	0.25	0.61
Encoder + gyro	**0.08**	**0.19**

Table 6.1 carries the hardware recommendation for the real car: an IMU alone barely helps, and the *wheel encoder* is what makes own-pose navigation work. In the closed loop, with encoder and gyro, the onboard estimate stays within 0.13 m of the truth at the median and 0.99 m at the 90th percentile over ten seeds (Figure 7.4).

6.4 The real-time budget on the Pi

Table 6.2. Measured on the Raspberry Pi 5 + Hailo-8L as the robot computer of the hardware-in-the-loop suite (38,207 frames; A*: 993 plans).

Stage (Raspberry Pi 5 + Hailo-8L)	median (ms)	p90 (ms)	samples
JPEG decode	1.0	1.6	38,207
Depth network on the NPU (288×512, int8)	35.1	35.8	38,207
CPU: scan + map + SLAM + navigator step	48.9	54.7	38,207
of which SLAM (own-pose runs)	18.5	23.2	20,183
of which navigator step	4.2	6.2	37,858
Hybrid-A* search (asynchronous worker)	109.9	546.8	993
Round trip, Pi ↔ simulator (incl. network)	88.0	93.9	38,207

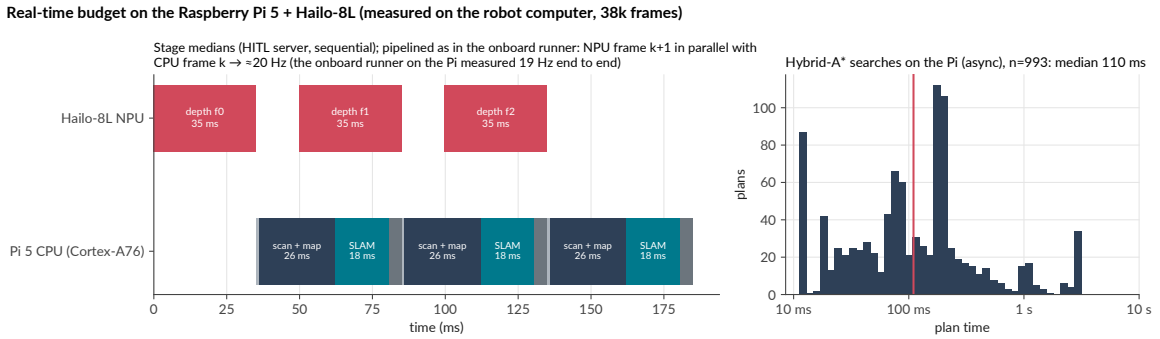


Figure 6.1. Left: per-frame stages (medians) and how the onboard runner overlaps them. Right: hybrid-A* search times on the Pi.

The depth network occupies the NPU for 35 ms per frame and the CPU stages take about 49 ms, so with the NPU working on frame $k+1$ while the CPU handles frame k the loop runs at about 20 Hz on simulator-sized frames. On real 1280×720 camera video the standalone onboard runner, with the deployed code, processes 300 frames in 18.1 s with ground-VO SLAM (16.5 Hz; feature detection and tracking at full resolution take 19 ms of it) and in

11.1 s without it (27 Hz, limited by the NPU). In the hardware-in-the-loop suite the round trip to the simulator adds the network link: 88 ms at the median. With the final deployed code the real board measured 35.0 ms on the NPU, 46.7 ms of CPU at the median (64.1 ms at the 90th percentile) and an 85 ms round trip, over 9,371 replies. The simulator applies each command only when it arrives, so this latency is part of every result in Chapter 7.

Sustained operation. Real time must also hold for hours, not only at the median. Between its last boot and the end of the real-board arms the Pi served 1000 hardware-in-the-loop episodes over 28 h of wall time (380,127 logged control steps). NPU time did not drift: 35.1 ms at the median and 35.9 ms at the 99th percentile. Afterwards, the firmware's sticky throttle register still read `0x0`: since boot there had been no under-voltage, no frequency capping and no thermal throttling.

Chapter 7

Hardware-in-the-loop evaluation

7.1 Set-up

World. NVIDIA Isaac Sim with Isaac Lab [98]. The yard is a Gaussian-splat reconstruction [55, 145] of a real yard, with a heightfield collider for its terrain. The robot is a model of the project's Traxxas with an Ackermann PWM action: ± 0.349 rad steering and up to 0.72 m/s. The camera is level, 504×280 with a 69.4° horizontal field of view, 0.13 m above the ground.

Obstacles. Eleven prop types from public asset libraries are placed on the rendered ground: an office chair, a cantilever chair, a school chair, a traffic cone, a cardboard box, a barrel, a trash can, a wet-floor sign, a crate and two posed people. Each has an invisible kinematic collider sized to what the car can actually hit:

- round bodies use their body radius;
- boxy props use their box;
- legged chairs use only the cells where their surface crosses the car's body band (4–30 cm). A solid cylinder had made an "invisible fence" between chair legs;
- people use a 0.25 m cylinder around feet and legs.

Scenes. Ten frozen layouts with 3–6 m between start and goal: chair, box offset, cone slalom, barrel gate, chair cluster, person on path, mixed clutter, crate wall with a gap, person and cones, and barrel. Scenes 07–09 were never seen by the depth network in training.

Protocol. Each configuration (an *arm*) runs all ten scenes for at least five seeds; a seed jitters the start by up to 0.15 m. An episode ends on success (car within 0.45 m of the goal), after 35 s, or if the car tips. It is *clean* if no point of the car's real 0.557×0.294 m footprint touched any collider at any time. Every frame is JPEG-encoded and sent to the robot computer, and the returned command is applied only when it arrives.



Figure 7.1. The deployed stack with the real Pi + Hailo-8L in the loop, on its own SLAM pose. Each panel shows the chase camera (top left), the robot's camera (top right), the depth computed on the Hailo-8L with obstacle pixels outlined and the commanded arc (bottom left), and the bird's-eye view with the SLAM path and the current plan (bottom right). Left: cone slalom, 6 s in. Right: people and cones, backing away from a cone seen at close range (this episode reached the goal with a brush).

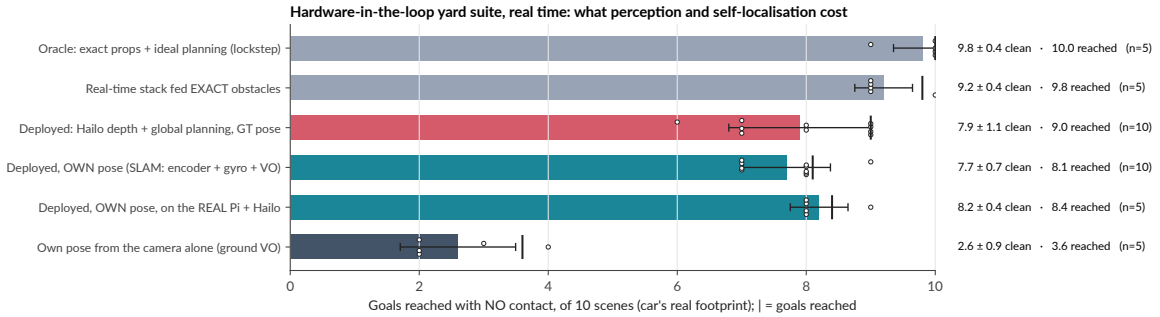


Figure 7.2. The error budget. Bars: clean goals of 10 (mean ± sd over seeds, dots = seeds); ticks: goals reached.

A faithful local brain. The real Pi and Hailo-8L ran the first arms. To run arms in parallel, the identical Pi code also runs on the laptop. The student executes in PyTorch with outputs identical to the chip (Pearson 1.0000 to its ONNX twin, which equals the HEF). Replies are padded to the Pi's measured round trip (84 ms median), and asynchronous plans are stretched by 2.35×, the measured Pi-to-laptop planner speed ratio. Replicating a Pi arm locally gave 6.6 vs 7.2 clean and 5.6 vs 5.6 on the stricter circle metric, within seed noise. The final, deployed configuration was then run again with the real board in the loop (own pose, seeds 0–4): 8.2 ± 0.4 clean against 7.6 for the local brain on the same seeds. The real round trip was 85 ms at the median (90th percentile 103 ms, 99th 113 ms, 9,371 replies), against the local brain's 84 ms. The local brain is, if anything, conservative.

7.2 Results

The error budget (Figure 7.2).

- An **oracle** that knows every prop exactly and plans in lockstep reaches **9.8** clean of 10: a perfect score is feasible on this suite.
- The same **real-time** stack fed **exact obstacles** reaches **9.2**. Latency, the follower and the ESC gate cost about 0.6.

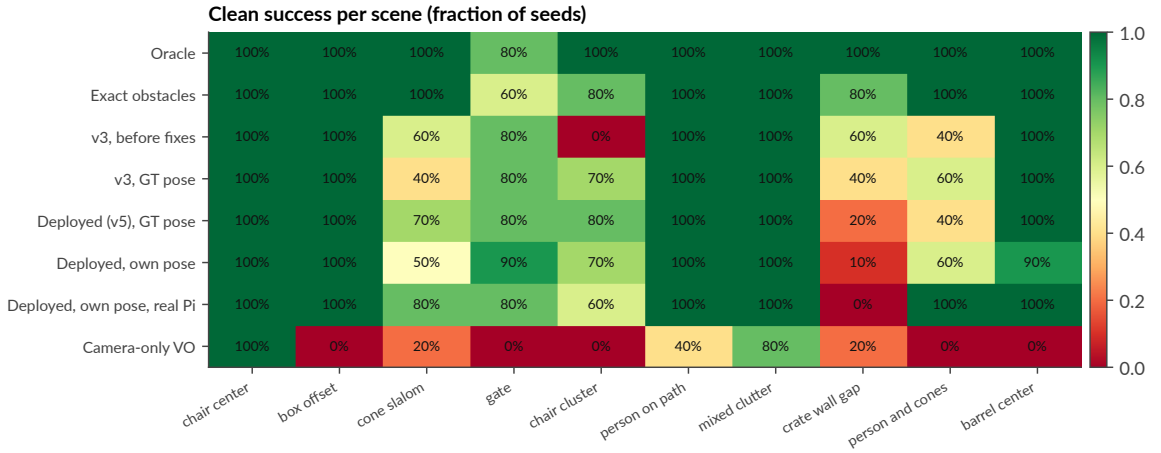


Figure 7.3. Clean success per scene. Remaining failures concentrate on the cone slalom, the gate, the chair cluster, the crate wall and the person-and-cones scene.

- With **depth from the Hailo-8L**, the deployed stack reaches **7.9** $\pm$ **1.1** (9.0 reached) over ten seeds. Perception costs about 1.3.
- On its **own pose** (SLAM with simulated encoder and gyro) it reaches **7.7** $\pm$ **0.7** (8.1 reached) over ten seeds, and **8.2** $\pm$ **0.4** with the real Pi and Hailo-8L in the loop (five seeds). Self-localisation costs little in clean goals (0.2) but 0.9 in goals reached. The failure mode changes with it: with simulator pose most misses reach the goal after a graze (11 of 21), while on own pose most are timeouts (18 of 23), usually after a brush, as drift misplaces the goal or mapped obstacles.
- With **camera-only odometry**, 2.6. A wheel encoder is not optional.

Student v3 or v5? The floor-weighted student v5 replaced v3 on the strength of ten seeds per arm, and the evidence is modest, so we state it exactly. With simulator pose the two are not resolved (7.9 $\pm$ 1.0 vs 7.9 $\pm$ 1.1, ten seeds each), with per-scene differences in both directions: v5 is better on the cone slalom (7 vs 4 of 10) and worse on the crate wall and the person-and-cones scene (2 vs 4, 4 vs 6). On the robot's own pose v5 is ahead, 7.7 $\pm$ 0.7 against 7.1 $\pm$ 0.9 (8.1 vs 7.4 reached), mainly through the chair cluster (7 vs 4) and person-and-cones (6 vs 4). The difference is not significant (permutation test $p \approx 0.17$). v5 also costs nothing: same architecture, same speed on the chip (27.1 FPS), int8 equal to float at Pearson 0.99993 (p5 0.99569), and the lower AbsRel on obstacle pixels of the two (0.197 vs 0.204). It is deployed because it is never measurably worse and is ahead on the pose the real car will use.

7.3 Failure analysis

Thin chair frames. Under the cantilever chair's seat the floor model took the seat for floor (Chapter 5). The horizon-anchored fix, together with the planner fixes below, took the chair cluster from 0/5 to 3/5 clean.

Table 7.1. Every evaluated arm (clean and reached goals of 10, mean ± sd over n seeds, scored on the car's real footprint).

Arm	Configuration	n	Clean (of 10)	Reached
Ceilings (no perception error)				
`o13`	Oracle: exact props, the Navigator's planner (lockstep)	5	9.8 ± 0.4	10.0 ± 0.0
`l21x`	Real-time stack, EXACT obstacles (no perception)	5	9.2 ± 0.4	9.8 ± 0.4
Hailo depth, simulator (GT) pose				
`p12rt`	Student v1 + global planner (Pi)	5	7.2 ± 1.3	7.8 ± 1.1
`p15rt`	Student v3 + global planner (Pi)	5	7.2 ± 1.1	9.0 ± 1.0
`l22rt`	v3 + PLAN_STEER 0.85 + straight-back retreat	5	7.4 ± 0.5	9.2 ± 0.4
`l26rt`	+ floor-from-horizon, pinned-retreat wedge, blocked fix	5	7.2 ± 1.3	8.8 ± 1.1
`l30rt`	+ escape-radius planning	5	6.8 ± 1.1	8.6 ± 1.1
`l31rt`	+ PLAN_STEER 0.75 + floating overhangs (the deployed planner, v3)	10	7.9 ± 1.0	9.0 ± 0.8
`l32rt`	+ base skirt on small obstacles (rejected)	5	7.2 ± 1.1	8.2 ± 0.8
`l35rt`	+ look first (neutral on own pose; not deployed)	5	8.0 ± 0.0	8.8 ± 0.8
`l36rt`	+ terrain-escape pass (neutral)	5	7.8 ± 1.1	9.2 ± 0.4
`l37rt`	+ faster follower (neutral)	5	7.4 ± 0.5	8.6 ± 0.9
`l38rt`	Student v5 (floor-weighted loss) **[deployed]**	10	7.9 ± 1.1	9.0 ± 0.9
`l39rt`	+ cone colour contacts (rejected)	5	7.8 ± 1.1	8.6 ± 1.1
Hailo depth, the robot's OWN pose				
`p12full`	Own pose: ground-plane VO only	5	2.6 ± 0.9	3.6 ± 0.5
`p16imu`	Own pose: v3 + sim encoder + gyro	5	6.0 ± 1.4	7.6 ± 1.1
`l22imu`	Own pose: l22 planner + encoder + gyro	5	6.2 ± 1.3	7.4 ± 1.3
`l31imu`	Own pose: deployed planner + encoder + gyro	5	7.0 ± 0.7	7.0 ± 0.7
`l34imu`	Own pose: + encoder-gated slip guard (deployed planner, v3)	10	7.1 ± 0.9	7.4 ± 1.0
`l35imu`	Own pose: + look first	10	6.8 ± 0.9	7.2 ± 0.8
`l37imu`	Own pose: + faster follower	5	7.2 ± 0.8	7.8 ± 1.1
`l38imu`	Own pose: student v5 **[deployed]**	10	7.7 ± 0.7	8.1 ± 0.7
`p38imu`	Own pose: student v5 on the REAL Pi + Hailo **[deployed]**	5	8.2 ± 0.4	8.4 ± 0.5
`l39imu`	Own pose: + cone colour	5	6.4 ± 0.5	7.2 ± 0.8

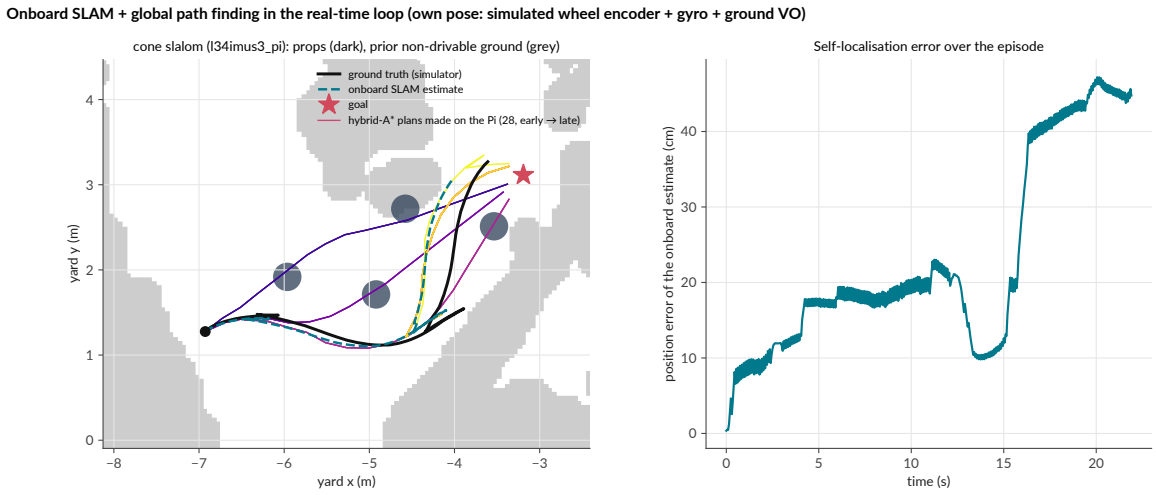


Figure 7.4. Onboard SLAM and global planning on the robot's own pose: every hybrid-A* plan the Pi made in one slalom episode, the SLAM estimate against the truth, and the localisation error over time. The car reached the goal despite 0.47 m peak drift.

Obstacles the camera has not seen. In the crate-wall scene the goal starts about 63° off the camera axis, and one crate lies 65–74° to the left, outside the ±34° view. It is never mapped while the car plans, so the first plans run through it. The car approaches, sees it, blocks, reverses and repeats. Penalising unseen space more widely did not help: every route to the goal crosses unseen space, and searches got 6–20× slower. A "look first" arc toward the goal raised the GT-pose score to 8.0 ± 0.0 but was neutral on own pose at ten seeds, so it is not deployed. This is an exploration problem, not a perception one.

Sides of small objects. The remaining contacts are almost all the car's front corner grazing the side of a cone, a person or a crate, which the scan places 5–15 cm too far out. This is the low-base limit of Section 5.5. Uniform margin increases were null because they also close the narrow gaps that other scenes require.

A regression we caused and fixed. One planner fix made the "path is blocked" check consistent with the planner, which ignored obstacle cells already overlapping the car. That removed an accidental safety net: the old, inconsistent check had been vetoing plans that ran through a crate face the bumper was touching. The crate-wall scene fell from 3/5 to 1/5. The proper fix ignores overlapped cells only within an escape radius of 0.3 m and treats them as obstacles beyond it, in both the planner and the check. A unit test reproduces the trap: the old code planned straight through a touched crate face, and the new code goes around it.

7.4 What did not help

The last row of Table 7.2 is worth stating plainly: **the student drives better than the simulator's own depth buffer**. The renderer's depth for a splat scene carries speckle and floaters near the floor (visible in Figure 4.3). The student, trained to the teacher's clean shape and

Table 7.2. Interventions that did not improve clean success at ≥5 seeds (clean goals of 10, against the baseline of the time on the same seeds, e.g. 7.6 for the v3 planner on seeds 0–4).

Intervention	Outcome	Clean
Base skirt: +6 cm on small mapped clusters	04 improves to 5/5 but the slalom closes	7.2 vs 7.6
Cone-colour contacts (orange mask)	cone gap closed offline; slalom timeouts in the loop	7.8 vs 7.6; own pose 6.4 vs 7.1
"Look first" arc toward an off-axis goal	neutral on own pose (10 seeds)	GT 8.0; own 6.8 vs 7.1
Third planning pass, terrain free within 1.5 m	fixes targeted timeouts, contacts move	7.8 vs 8.0
Faster follower (0.6 / 0.45 throttle)	own-pose drift p90 0.99→1.63 m	7.4 vs 7.6
Uniform blocked-path margin 0.04 m	extra re-plans stop the car	6.6 vs 7.4
Mapping low hits beside tall ones	—	7.0 vs 7.2
2× supersampled camera pixels	—	own pose 2.6 vs 2.6
Latency-compensated follower	worse on a synthetic S-curve; reverted	—
Renderer GT depth instead of the student	GT depth has speckle and floaters (GT arm ran lockstep, so not a matched pair)	5.2 vs 6.8

then to that ground truth, produces a cleaner signal for ground-contact ranging. The comparison is conservative in the renderer's favour: its arm ran in lockstep, without the round-trip latency the student's arm had to absorb.

7.5 Statistics

Seed-to-seed standard deviations are about one goal in ten, so differences below about one goal at $n = 5$ are not resolvable. We report means, standard deviations and every seed, and extend arms to ten seeds when a decision depends on them. Contacts are scored on the car's real rectangle rather than the simulator's 0.22 m contact circle, which over-reports side brushes. A small number of seeded starts are thrown by the physics settle into contact with a prop; excluding contacts already present before the first command moves the v3 GT-pose score from 7.9 to 8.0 and leaves both deployed v5 arms unchanged. We report the stricter number.

Chapter 8

The real world: one drive, every sensor, every method

The simulator answers questions with exact ground truth; the real world answers whether those answers survive. This chapter takes one recorded drive of the real car through a sunlit park, every stream of the RealSense D435 plus the Hailo-8L output of the deployed student, and asks three questions of it: which depth is better, how each depth source behaves inside SLAM, and how our onboard SLAM compares with classic and recent systems. Where the real drive cannot answer (it has no ground truth), the same questions are put to the hardware-in-the-loop simulator with a simulated lidar, a simulated D435 stereo pair and exact poses.

8.1 The recording instrument

What is recorded. One button in the robot's web cockpit (*REC ALL*) records every stream the RealSense D435 offers, the onboard network's output and the operator's commands, losslessly and on the robot itself: the 640×360 colour image the network sees (JPEG, quality 95); the on-chip stereo depth at 848×480 (16-bit PNG, millimetres), Intel's recommended D435 depth mode and the same ≈16:9 aspect as the colour image; both infrared images of the stereo pair at 848×480 (lossless PNG), so every stereo algorithm can be re-run offline on exactly the images the camera matched; the Hailo-8L output of the deployed student for every frame it processed (float16, mapped to its exact source colour frame); and the 20 Hz steering and throttle commands. The camera holds a plain D435 (no inertial unit) and the car carries no lidar. The data rate is about 32 MB/s, far beyond the robot's wireless link, so sessions are written to the Raspberry Pi 5's SD card and copied to the laptop afterwards, over Ethernet at ≈76 MB/s. A session is deleted from the robot only after an `rsync` checksum pass has found the laptop copy byte-identical.

Throughput. On the Raspberry Pi 5, PNG encoding costs 21 ms per depth frame and 14.5 ms per infrared image (≈1.5 cores at 30 fps) and the SD card sustains 73–80 MB/s. The first 46 s test lost 39 infrared and 22 colour frames in a single burst. The cause was the kernel's write-back policy: it lets about 1 GB of dirty pages accumulate and then flushes them at once, blocking the writers for 2–3 s. Capping dirty memory at 256 MB (background write-back from 64 MB) removed the stall: in the repeated 45 s test the recorder dropped no

frame it received (checked by sequence numbers, which defect 2 shows cannot see drops inside the camera), and the onboard network kept 25.8 fps (27 without recording).

Three defects found in the first real session, and how each was worked around. The first drive through a park exposed three faults in the instrument. All three are reported because each would have silently corrupted every comparison downstream.

1. *Every depth and infrared frame was byte-shifted.* The capture tool prints its format description to the same pipe that carries the frames. Every fixed-size chunk was therefore offset by that text: 365 bytes for depth, an odd number that scrambles every 16-bit value, and 380 bytes for the infrared pair, which wraps each image by 190 pixels. The header sits at the start of the first chunk, and lossless storage meant the original stream was intact, so each frame was rebuilt exactly as $F_k = c_k[h{:}] \,\|\, c_{k+1}[{:}h]$. 6,177 depth and 6,145 infrared frames were recovered, with only the last frame of each missing its final h bytes. The capture now sets the format in a separate call and also strips any leading text.

2. *The stereo stream's timestamps drift against its content.* The depth and infrared timestamps advance by a perfect 33.4 ms per frame with contiguous sequence numbers. Cross-correlating each stream's yaw rate with the others, however, shows the content running ahead of the stamps: by 1.2 s after 15 s and 9.6 s after 135 s, with stall-like steps. The colour stream follows the recorded steering at a steady 0.13–0.22 s actuation lag (r 0.83–0.96 in 12 of 13 windows of 15 s), so it is the stereo stamps that are wrong. The camera delivered about 8 % fewer frames than it stamped, and the stamps, synthesised from its own clock, hide the gaps. Pairing colour and stereo frames by timestamp would have mismatched them by up to 12 s. That session is resynchronised from image content: the residual against the colour stream's yaw is a median 12.5 ms (p90 31.7 ms, maximum 208 ms), and the 26 of 388 benchmark frames that fall in windows off by more than one frame are flagged and excluded. New sessions record each frame's host arrival time and pair on that.

3. *A power cut after recording.* The Raspberry Pi 5 lost power within ≈30 s of the operator stopping the recording. The kernel had not yet written the session summaries (left at 0 bytes) or the last frames to disk. The usable window, where every stream has surviving timestamps, is 1.2–203.5 s of the 219 s recorded (the SLAM window of Section 8.4, from the first stereo frame to the end, is 216 s). The instrument now forces a sync when recording stops and writes back dirty data within 5 s rather than 30 s, so a cut costs at most ≈5 s. The robot's system log now survives reboots, so the next power loss leaves evidence.

A measurement system for a thesis has to be measured too. All three defects passed a first "every stream checks out" inspection that looked only at frame counts and rates. They were caught by looking at the images and by cross-checking streams against each other, never by trusting one stream's own metadata.

8.2 Which depth is better: a 1000-point benchmark

Reference without ground truth. The drive has no lidar and no surveyed scene, so the reference is built from agreement between three different stereo matchers run on the same infrared pair: the D435's on-chip matcher, OpenCV SGBM and Fast-FoundationStereo. A

pixel is a reference point only where all three agree within $2\,\% + 1\,\%/\mathrm{m}$, at 0.3–3 m, on a flat 5×5 neighbourhood, registered into the colour view with the camera's factory calibration (read from the device, below). From these, 1,000 points are frozen once: stratified over five distance bins (217/228/215/165/175 points) and a 3×3 image grid, at most four per frame, from 309 frames, inside the image region every model sees. Every monocular method is scored on these identical points.

Calibration. The drive was recorded before the D435's factory calibration could be read, and the first version of this benchmark registered the points with the datasheet's nominal values (principal points at the image centres, a pure 15 mm baseline to the colour sensor). The factory values, read later from the same unit (serial 825312071386, `rs-enumerate-devices -c`), put the infrared principal point 10 px lower and the colour one 6 px right and down, and rotate the colour sensor by 0.65°. Together these move every registered point by a median 14.5 px (p95 16.4) in the 640×360 colour image. Two checks say the factory values belong to this recording. Depth edges registered with them land a median 6.7 px from the colour image's edges, against 12.0 px with the nominal values, closer on all 20 frames with enough depth edges to score (sign test $p \approx 10^{-6}$). And on the *same* physical points (the 894 nominal points that stay inside the shared crop), the registration alone lowers the AbsRel of 17 of the 19 networks by 10–47 %, while the three stereo scores, computed in the infrared frame, do not move; the two weakest, SC-DepthV3 and FastDepth, barely change ($+0.5$ and $-4\,\%$). Sharp models gain most (Depth Anything V2 AbsRel 0.24→0.13, −44 to −47 %; Depth Pro 0.32→0.19; ours 0.27→0.20, −24 %; the zoo Depth Anything V2-S 0.27→0.23, −13 %): consistent with misregistration having blurred the reference in a way that flattered the low-resolution chip models. All numbers below use the factory calibration and a point set rebuilt with it; the nominal results are kept beside them (`*_nominal.json`) as a record.

Alignment. Relative models are aligned per frame on held-out consensus points of that frame, never on the scored points: disparity outputs by scale and shift in $1/Z$, Depth Anything 3's affine-invariant depth by scale and shift in depth, up-to-scale depth by a median scale; predictions are clipped to 0.05–10 m. A second score fits one scale (and shift) for the whole drive, on this drive's own consensus points. Comparisons are paired: a frame-cluster bootstrap gives the confidence interval of each difference and a sign-flip test its p-value. Frames are 0.5 s apart and not independent, so every key comparison is also tested with 10 s blocks of the drive as the unit (block length fixed before looking; 21 blocks, 14 on the obstacle strata). The decision statistic is the exact block sign-flip p (all 2^{14} or 2^{21} sign patterns). Intervals are frame-cluster bootstrap CIs unless marked "blocks"; with this few blocks a percentile block bootstrap can under-cover, so both are given for the headline comparisons.

Results (Table 8.1). On these points the student, as it ran on the chip during the drive, reaches AbsRel 0.262; replaying the recorded frames through the chip gives 0.258, and so does the float twin. The live run is resolvably worse than the chip replay, by $+0.004$ (95 % CI $[0.001, 0.006]$, $p = 0.005$; blocks $p = 0.001$), though not on the set rebuilt without FastFoundationStereo ($+0.001$, $[-0.004, +0.005]$, $p = 0.68$). Against the Hailo model-zoo Depth Anything V2-S compiled for the same chip the student is *not resolvably different*: paired difference $+0.017$, 95 % CI $[-0.060, +0.118]$, $p = 0.70$ (10 s blocks, the exact sign-flip test inverted: $[-0.040, +0.075]$, $p = 0.66$), and on the symmetric error $|\log(\hat{z}/z)|$ $+0.009$ ($[-0.012, +0.032]$, $p = 0.42$). The interval bounds any advantage of ours at 0.060 AbsRel (0.040 with blocks),

Table 8.1. The real 1000-point benchmark (park, full sun; one drive, one run; factory calibration). AbsRel and δ_1 with each output's own per-frame alignment, and AbsRel with one alignment for the whole drive. The stereo rows are not comparable with the monocular rows: each stereo method is scored leave-one-out, in the infrared frame (no registration or resynchronisation error), on its own point set from the *other two* matchers' consensus with its holes excluded (n = 950 / 830 / 1000). They measure agreement with the other matchers on the same images.

Method	AbsRel	δ_1	AbsRel, one scale
Depth Anything 3 S / B / L / mono-L (GPU)	0.101–0.115	0.86–0.90	0.36–0.42
Depth Anything V2 S / B / L (GPU)	0.160–0.181	0.82–0.84	0.42–0.43
Depth Pro / MoGe-3 / MoGe-2 (GPU, aligned)	0.202 / 0.205 / 0.207	0.645 / 0.625 / 0.628	0.402 / 0.261 / 0.262
UniDepthV2-L / DA3-Metric-L (GPU, aligned)	0.222 / 0.222	0.614 / 0.580	0.305 / 0.297
Hailo model-zoo Depth Anything V2-S (on chip)	0.234	0.736	0.432
Ours, chip replay / float twin	0.258 / 0.258	0.739 / 0.751	0.413 / 0.410
Ours, live on the Hailo-8L during the drive	0.262	0.730	0.410
Our int8 Depth Anything V2-S / SC-DepthV3 / FastDepth (on chip)	0.443 / 0.392 / 0.394	0.580 / 0.219 / 0.227	0.439 / 0.435 / 0.415
D435 on-chip stereo (5 % of points without depth)	0.009	0.999	–
OpenCV SGBM (17 % without depth)	0.010	0.995	–
Fast-FoundationStereo (0 % without depth)	0.040	0.970	–

and ours may be up to 0.118 worse. The simulator's advantage (0.269 against 0.354 all-pixel AbsRel on held-out frames of the yard the student was fine-tuned on, Chapter 4, a different quantity) was 0.085 absolute, 24 % relative: on this set it is excluded in absolute terms. In relative terms the paired test runs on frame-averaged errors (ours 0.232, zoo 0.215 per frame): inverting the exact block test on their ratio bounds our advantage at 23 % ($p = 0.039$ for 24 %; `depth/ratio_bound.json`), while a frame bootstrap of the ratio reaches 27 % and does not exclude it. In relative terms the claim rests on a one-point margin of the block test and carries no weight. On the set rebuilt without Fast-FoundationStereo (below) the interval does not exclude it. The sign of the estimate also depends on the point draw: on the 894 points kept from the nominal set and re-registered, ours against the zoo model is -0.023 (CI $[-0.075, +0.021]$, $p = 0.37$); a null either way. The points are mostly ground. On a separate obstacle stratum (consensus points at least 0.15 m above each frame's RANSAC floor plane: trunks, bench backs, bushes, people, and some rising far lawn; 319 points on 87 frames, `depth_obst/`), the mean AbsRel is ruled by a few extrapolation blow-ups (the zoo Depth Anything V2-S reaches 2.2). On $|\log(\hat{z}/z)|$, where the student and its own chip replay differ by only 0.007, ours *leans* ahead of the zoo Depth Anything V2-S: -0.165 (95 % CI $[-0.290, -0.050]$, $p = 0.012$ with frames as units; with 10 s blocks, exact $p = 0.046$, inverted-test CI $[-0.311, -0.002]$). Holm-corrected it is 0.14 over the five paired tests on this metric (which include two fidelity checks) and 0.046 over the three competitor tests: the ver-

dict depends on the family, so the stratum is exploratory. On the D435+SGBM consensus (396 points on 101 frames, `depth_noffs_obst/`) the direction is the same, -0.123 (blocks: $p = 0.055$). The stratum was added after the main null, at a review's request; its metric was chosen on the nominal build, before this re-score, and the calibration by an edge test that never looks at a network. The lean sits in the tail, fewer large errors; the median barely differs (0.232 against 0.240; 0.369 against 0.346 on the D435+SGBM set; untested). With one scale for the whole drive the direction reverses on the D435+SGBM set: ours is worse by $+0.334$ (inverted block CI $[0.019, 0.659]$, $p = 0.040$; Holm 0.058 over that set's three per-drive tests, so equally exploratory), and on the three-matcher set not resolved ($+0.038$). So the simulator's obstacle advantage is not shown on this drive. Depth Anything V2-L is better than ours there by 0.153 (blocks: inverted CI $[0.023, 0.267]$, $p = 0.012$; on the D435+SGBM set $+0.012$, not resolved) and DA3-L by 0.184 (CI $[0.128, 0.240]$); in mean AbsRel MoGe-3 and Depth Pro lead ours there ($+0.21$, $+0.23$), not resolvably with blocks ($p = 0.27$, 0.19). The 0.15 m threshold was fixed when the stratum was defined, before its first score. Of the 339 frames that pass synchronisation, 30 contribute none of the 1,000 points (no consensus point inside the shared crop, or not drawn by the stratified sampler), hence 309. The student beats our int8 Depth Anything V2-S build, SC-DepthV3 and FastDepth (block $p < 0.001, 0.009, 0.014$), though the last two output up-to-scale depth and get a one-parameter median scale where ours gets scale and shift. It trails its teacher Depth Anything V2-L by 0.091 in the table and by 0.085 as a frame-averaged paired difference (95 % CI $[0.047, 0.141]$, $p < 0.001$). Depth Anything 3 is the best monocular family by far. Aligned per frame, every metric model now scores better than ours (0.20–0.22 against 0.262), but none resolvably (MoGe-3 and Depth Pro: $+0.044$, CIs $[-0.029, +0.155]$ and $[-0.025, +0.148]$). Raw, every metric model *overestimates* distance from the robot's 16 cm viewpoint (median prediction over truth $\times 3.5$ MoGe-3, $\times 3.3$ MoGe-2, $\times 2.4$ DA3-Metric, $\times 2.0$ Depth Pro, $\times 1.7$ UniDepthV2), plausibly because they were trained on cameras at human height. With one scale for the whole drive, fitted on this drive's own consensus points, metric MoGe-3 and MoGe-2 (0.26) have a lower error than every relative network (0.36–0.44; not paired-tested); ours (0.410) sits among them, ahead of the zoo Depth Anything V2-S (0.432) and Depth Anything V2 (0.42–0.43) but not resolvably (against the zoo -0.040, 95 % CI $[-0.106, +0.061]$, $p = 0.49$). Every relative Depth Anything V2- and Depth Anything 3 network loses 0.15–0.32 under one scale per drive (ours 0.15, the zoo Depth Anything V2-S 0.20, Depth Anything V2 0.24–0.27, Depth Anything 3 0.25–0.32), except our int8 Depth Anything V2-S build (-0.005), which is poor under either alignment, as are SC-DepthV3 and FastDepth ($+0.04$, $+0.02$); the metric-trained DA3-Metric loses 0.07. The output scale of relative networks moves from frame to frame. Relative networks need a per-frame scale; the onboard stack takes it from the camera height (Chapter 5), which this benchmark does not score.

What the set cannot say. The points lie where stereo is reliable: textured surfaces closer than 3 m, mostly ground and near objects. Sky, glare, and objects closer than the stereo's minimum range, which the network still sees and the stereo does not, are absent by construction. The three matchers share the images and the rectification, so a common-mode error there is invisible to the consensus. SGBM and Fast-FoundationStereo convert disparity with $f_x B = 21.42$ px m fitted to the D435's own depth; the factory value is 0.8 % lower (21.24), a sub-pixel disparity bias in SGBM. A per-frame alignment absorbs any global scale of the reference, so only which points pass the consensus could change. Fast-FoundationStereo is distilled from FoundationStereo, whose backbone includes Depth Anything V2 features; no Depth Anything V2 runs at its inference, and as one of three voters it can only veto points, never admit one, but the set may lean toward points where a Depth Anything V2-like prior

is right. Rebuilding the set without it (consensus of the D435 and SGBM only, a new frozen 1,000 points, 321 frames) does not change the verdict against the zoo model: ours against the zoo Depth Anything V2-S is -0.061 (block $p = 0.078$, inverted-test CI $[-0.130, +0.007]$; frames: $[-0.132, +0.001]$: not significant), a mean carried by the zoo model's blow-ups, since on $|\log(\hat{z}/z)|$ the difference is not resolved (-0.006, CI $[-0.030, +0.017]$); one scale per drive -0.010 ($[-0.071, +0.065]$, $p = 0.81$); against Depth Anything V2-L $+0.028$ ($[-0.013, +0.066]$, $p = 0.18$), not resolved. A first version scored the stereo matchers on points from a three-way consensus they had each voted in; that was circular and was replaced by the leave-one-out scoring above.

8.3 Same poses, swap the depth

To isolate the depth from everything else, both sources are run through the onboard pseudo-lidar (one ray per image column, ending at the first obstacle) and integrated into occupancy maps at the *same* poses: the independent RTAB-Map reference of Section 8.4. Rays are matched by bearing, never by column index (stereo holes drop columns). In both sources a ray's range comes from ground-contact geometry, the image row where the obstacle meets the floor plus the camera height, so this compares where each source places an obstacle's base, not depth values; at 3 m one pixel row is ≈0.12 m. Both run the onboard geometry as deployed, with nominal colour intrinsics (69.4° field of view, principal point at the image centre, level camera). The factory principal point sits 5.6 px lower; since a ground-contact range is set by the row offset from the horizon (≈25 px at 3 m), that shift changes each range by about 11 % at 1.5 m and 23 % at 3 m in the same direction for both sources, so the relative gap moves too. An uncorrected pitch of a few degrees would move it more. The 25 % is a property of the deployed geometry, not a calibrated range error (not re-scored with the factory principal point). Two regimes emerge. Beyond 1 m, where both sources report an obstacle, the network's range is within a median 25 % of the stereo's per frame (IQR 15–37 %, 931 frames; pooled over 28,616 rays, 29 %, IQR 14–48 %); signed, the median is -7 %, the network placing obstacle bases a little nearer, similar in size to the benchmark's AbsRel, though a different quantity. Inside 1 m both sources call grass an obstacle, in different places: the stereo resolves single tufts from 16 cm and marks a median 63 % of the image columns blocked, the network 41 %, and the network sometimes draws a false band across motion-blurred lawn. Neither is right on grass, which this car drives straight over. The stereo was blind (sun-saturated infrared) on 0.1 % of frames. Seen from above, of the 10 cm cells in which either map holds a point at least 0.15 m above the ground, the stereo marks 86 % and the network 39 % (both 25 %): a descriptive picture (the cell size and height threshold were not varied, and the network's map carries its floor-fit scale). The split-screen video on the project page shows the two maps being built side by side.

8.4 SLAM on the real drive

Reference and protocol. Without ground truth, the yardstick is an independent metric system: RTAB-Map RGB-D on the D435's left infrared image and its native stereo depth, one graph over 97 % of the drive with four loop closures and 47 odometry resets. Every trajectory is put on the colour camera's clock (infrared-clock runs re-timed through the content resynchronisation of Section 8.1), aligned to the reference over its own overlap ($\mathrm{SE}(2)$ for metric methods; $\mathrm{Sim}(2)$, flagged, for monocular ones) and scored by RMS error and by

$skill = 1 - \mathrm{RMS}/\mathrm{RMS}_{\mathrm{parked}}$, where the null track sits still at the reference's centroid. Skill measures agreement with RTAB-Map, not accuracy; it is one drive and one run per method, except ORB-SLAM3 stereo, DROID-SLAM and VGGT-SLAM, run three times each. The two alignments are not like for like: under $\mathrm{Sim}(2)$ a scale of zero reproduces the parked track, so skill cannot fall below 0, while under $\mathrm{SE}(2)$ it can. Scored with the same $\mathrm{Sim}(2)$ alignment the monocular methods get, our stack as deployed reaches $+0.10$ and the commands alone $+0.08$ (`slam_sim2_skill.json`; fitted scales 0.40 and 0.29, both tracks shrunk toward standing still), against VGGT-SLAM's $+0.80$. The reference itself is not exactly repeatable: three re-runs (one with the nominal, two with the factory infrared intrinsics) differ from it by 0.64–0.79 m RMS and from each other by 0.33–0.49 m. Re-scoring every method against each of these four reference runs moves no skill by more than 0.06 (MASt3R-SLAM most, $+0.90$ to $+0.96$) and changes no ranking among the methods that track the whole drive (partial-coverage ones swap places). Each baseline that runs on TUM fr1/desk first reproduces its published accuracy there, so a broken build cannot pass for a weak method; ORB-SLAM3 monocular failed one of four validation runs, ORB-SLAM3 stereo was not validated (fr1 has no stereo pair) and neither was Glob3R (a global structure-from-motion system, not run on fr1), and CoreSLAM was checked on its own bundled log.

Results. Methods that use the colour camera alone agree with the reference: VGGT-SLAM 2.0 scores skill $+0.80$ (the same in all three runs: its trajectories are bit-identical) on keyframes spanning 99 % of the drive (351 keyframes; 117 s of the drive lie in tracking gaps by the run's own rule, filled by linear interpolation, and scored at its own keyframes only it reaches $+0.79$), the 2026 global structure-from-motion system Glob3R $+0.72$ (863 frames at 5 Hz), DROID-SLAM $+0.27 \pm 0.19$ over three runs ($+0.39$, $+0.36$, $+0.06$) and DPVO $+0.29$, all with their scale fitted; DROID-SLAM cannot be ranked against DPV-SLAM ($+0.39$) or DPVO. They are the only checks independent of the reference's inputs. ORB-SLAM3 stereo and RGB-D score $+0.885$ to $+0.892$ on the 16–23 % they track before fragmenting into 7–12 maps (stereo over three runs: $+0.879 \pm 0.013$ on 13–19 %), but they share the reference's inputs (the same infrared images, the same D435 depth), so their agreement is partly common input; and a skill on a partial overlap cannot be ranked against one on the full drive, because the parked null shrinks with the segment (MASt3R-SLAM scores $+0.90$ on 1.1 s). The classic CoreSLAM on a RealSense pseudo-scan scores about zero; ORB-SLAM3 monocular and MASt3R-SLAM lose tracking within seconds.

The commands alone, with no vision, score -0.31. Our stack as deployed scores -0.11 on the network's depth and measures the path at $0.43\times$ the reference (the commands' own path, from a speed model calibrated in the simulator, is $0.27\times$). Its 0.20 lead over the commands is not evidence that our odometry beats them: under the $\mathrm{Sim}(2)$ alignment it shrinks to 0.02 ($+0.098$ against $+0.078$), and with the physical gate below our stack trails the commands under $\mathrm{Sim}(2)$ too ($+0.012$ and $+0.024$ against $+0.078$). The $\mathrm{SE}(2)$ lead comes from paths shrunk toward standing still. Both are single runs of a deterministic replay (three runs of ours give a bit-identical scored trajectory), so the 0.20 has no run-to-run spread; how it varies from drive to drive, one drive cannot say. Its $+0.03$ on the RealSense's depth ($+0.12$ with the nominal registration) is not attributed to depth: the two arms run the same visual odometry on different frame sets (5,612 and 5,882 frames). As deployed, depth barely enters our trajectory: switching the scan match off changes neither arm, with the deployed gate (network -0.097 against -0.111 with it, RealSense $+0.043$ against $+0.029$) or with the physical gate below (-0.615 against -0.618; -0.381 against -0.382). The gaps between the two arms are therefore the odometry on different frames, not the depth, and this drive can-

not rank the two depths inside our SLAM; the simulator can (Section 8.5). The cause of the $0.43\times$ path is a constant: the ground visual odometry's plausibility gate was $1.5\times$ the *simulated* car's top speed and rejected 23 % of frames, exactly the fast ones. With a physical gate (5 m/s, 3 rad/s, chosen without looking at the reference) the path comes out at $1.08\times$ the reference's, the only input-independent check of the reference's metric scale, but heading errors accumulate and skill falls to -0.38 to -0.62, worse than the commands alone (SE(2); under Sim(2) also below them, above). Ground-plane odometry from a 16 cm camera on grass is the weak link; uncorrected roll of -2 to $-5°$ is the prime suspect (not isolated), and an inertial heading is the planned, untested fix. As deployed our loop closure closes nothing: 178 of 205 revisits face the opposite way, which a forward 69° camera cannot match, and the 27 same-direction revisits did not close either; with the physical gate the network arm closes one loop and the RealSense arm none. RTAB-Map closes four loops and VGGT-SLAM nine, by appearance.

Two errors of our own were published and corrected the same day: the first reference was our own stack on RealSense depth, which shares its odometry with every "ours" arm (agreement by construction), and visual-SLAM trajectories were first projected from the optical frame with the wrong axes, which made two working systems look collapsed. Both are recorded in the findings log with the corrections.

8.5 SLAM with ground truth: which depth is better inside SLAM?

The real drive cannot say which depth makes the better map: it has no ground truth, and our odometry fails upstream of the depth. The hardware-in-the-loop simulator can. Its yard scene was given three sensors the real car has or could have (a D435-geometry infrared pair at the true 50 mm baseline, matched by OpenCV SGBM; a planar lidar, three rings of 401 beams over $\pm 100°$, 0.1–12 m; the renderer's exact depth) and every pose is logged. A scripted controller drives a 7.9 m stadium loop twice in the same direction, once each under three lighting presets (noon, evening, dusk), on the identical ground-truth trajectory (16.6 m, 884 frames). Every depth source then goes through our onboard SLAM, and each trajectory is scored by its RMS error after a rigid SE(2) fit to the true camera track (ATE). Each accepted loop closure is checked against the true relative pose between its two keyframes. One seed, one trajectory, three sessions: the spread below is lighting plus render nondeterminism (for lidar and exact depth, render nondeterminism alone), not seeds or routes. Our network is in-distribution in every arm: student v5 was fine-tuned on exact disparity rendered in this same yard.

The deployed front end ignores the depth. With visual odometry tracking, the scan match never moves the pose (0.2 % of frames at most, for every source). Its motion prior allows 2 cm on a 5 cm grid, so a one-cell correction costs more score than the best possible match can gain. Front-end error is 0.60 m whatever the depth. Depth reaches the trajectory only through loop closure, and the closures it produces are mostly imprecise.

The bar: no vision at all. The commands alone score 0.107 m (noon only, one session; dusk SGBM's fallback reproduces it); loop closure on exact-depth floor-contact scans makes it worse, 0.161 m (both closures false). The simulated car executes its commands without slip, so every scan-odometry arm measures how much a depth source corrupts a near-exact prior.

Table 8.2. Our SLAM on simulated ground truth (ATE after loop closure, m; mean ± sd over the three sessions of one trajectory). *Scan*: odometry from scan matching (visual odometry off; VO = the deployed ground visual odometry). Plane scan: read off the depth image's horizontal plane, with the per-frame floor-fit scale, that scale smoothed over ∼20 frames, or the true metric scale. In brackets: loop closures accepted over the sessions (of which > 0.3 m off the truth). †SGBM, $n = 2$ (noon, evening): at dusk it gave no usable scan, the arm fell back to the commands (ATE 0.107, dead reckoning) and is excluded. *Ours, chip*: the same network's int8 output from the Pi's Hailo-8L, replayed offline over the session frames.

Front end and scan	Exact depth	SGBM stereo	Ours, float	Ours, chip
Commands only	0.107; with closures matched on exact-depth floor-contact scans 0.161 [2 (2)]			
(noon)				
Deployed:	0.671±0.003 [3 (3)]	0.543±0.078 [11 (7)]	0.532±0.062 [5 (4)]	0.561±0.077 [2 (1)]
ground VO				
Floor-contact scan	0.844±0.034 [0]	0.813±0.107 [2 (2)]	0.650±0.327 [1 (1)]	0.489±0.278 [4 (2)]
Plane scan, floor fit	0.501±0.069 [11 (5)]	0.356, 0.609† [1 (0)]	0.433±0.090 [4 (1)]	0.523±0.166 [3 (2)]
... fit smoothed	0.415±0.129 [9 (0)]	0.827, 0.612† [1 (1)]	0.732±0.057 [2 (2)]	0.469±0.228 [5 (1)]
... *true* scale	**0.074±0.003** [30 (0)]	0.195±0.048 [10 (0)]	–	–
Lidar ring as scan	**0.083±0.013** [22 (0)]; under the deployed visual odometry 0.566±0.047 [2 (0)]			

Only the lidar and true-scale exact depth beat it; true-scale SGBM (0.162–0.250 m) is worse than the commands in every session, and so is every other camera arm.

When the depth drives the odometry. With visual odometry off, scan matching becomes the odometry and the sources separate (Table 8.2). The lidar reaches 0.083 m with 22 closures, every one within 0.3 m of the truth. Through the deployed floor-contact pseudo-scan, *exact* depth (0.84 m) is no better than our network (0.65 m). Three controls locate the cause:

1. Reading the scan from the depth image's horizontal plane, like a planar lidar, helps exact depth in every session (0.84→0.50 m) and SGBM in both sessions where it produced a scan. For ours the mean gain (0.65→0.43 m) comes from one session (dusk, 1.01→0.35 m); evening gets worse (0.38→0.42 m).

2. Giving that scan the *true* metric scale instead of the per-frame floor fit brings exact depth to 0.074 m, in line with the lidar. The simulated lidar is the same renderer's noise-free depth sampled on one ring, so that equality is a pipeline consistency check, not evidence that camera depth matches a real lidar. SGBM's simulated passive-stereo noise (no projector) costs a factor of 2.6 (0.195 m).

3. Smoothing the floor-fit scale over time does not reliably help exact depth (0.42 m on average, but worse at noon, 0.47→0.56 m) and for the network its sign is not resolved: with

the float twin it hurts in all three sessions (0.43→0.73 m), with the chip's int8 output it helps at noon and evening (0.71→0.30, 0.47→0.37 m) and hurts at dusk (0.39→0.73 m).

Why smoothing cannot close the gap to the true scale: on exact depth the per-frame floor fit places a true 1 m at a median 0.999× and a true 2 m at 1.067× (3 m: 1.13×, heavy-tailed, mean 1.33×), identically in the three sessions, and at 1–2 m the error is slow, not jittery: at 2 m its sd across frames is 0.32 but frame to frame only 0.042 (1 m: 0.122 against 0.013). The fit drifts along the route, plausibly with the yard's uneven ground (untested), and averaging in time cannot remove a drift (one simulated loop, one seed).

The real chip in the loop, and what this SLAM resolves. The student's int8 output from the Pi's Hailo-8L, computed offline on the same session frames (the frames replayed through the chip on the Pi, then the SLAM run on the laptop; not a closed loop), agrees with the float twin at a median $r = 0.9999$ per frame at noon, 0.9998 in the evening and 0.9988 at dusk (all 2,652 frames; dusk minimum 0.88, `hitl_slam/chip_fidelity.json`). The floor-fit scale the SLAM takes from each frame differs more: chip over float by a median $+0.4$, $+0.6$ and -1.4 % (dusk 5–95 %: -8 to $+5$ %). Those differences move the per-session ATE by a median 0.13 m and up to 0.48 m, over 12 session pairs (four front ends, three sessions; column *Ours, chip* of Table 8.2): a SLAM ruled by its scale turns scale differences of a few per cent into tenths of a metre, so differences between camera arms that small are not resolved. What survives is the large effect: the true scale (0.074 m) and the lidar (0.083 m) against every arm without the true scale (means 0.42–0.84 m; true-scale SGBM 0.195 m sits between), and the commands alone (0.107 m) against every camera arm without the true scale (the best single session, the chip's pseudo-scan at noon, 0.169 m). Exact depth's floor-contact pseudo-scan is no better than the network's on either build (0.84 against 0.65 float and 0.49 chip).

In our 2D SLAM on one simulated loop, giving exact depth its true metric scale instead of the per-frame floor-fit scale cuts ATE 6.8× after loop closure (0.501±0.069→0.074±0.003 m, 6.3–7.4× per session; the front end alone 3.1×). Smoothing the floor-fit scale in time removes its jitter but not its slow drift, and does not recover the loss: the SLAM needs a scale free of slow drift. Two other explanations are untested: a steady but constantly biased scale, and the floor fit's distance-dependent bias (1.00× at 1 m, 1.07× at 2 m, 1.13× at 3 m). The floor-fit scale is the right tool for obstacle avoidance (Chapter 5) and the wrong one for this SLAM. Of the scale sources a camera-depth SLAM on this car could use, only stereo was tested here (0.195±0.048 m); an inertial unit and a metric model are untested, and a single per-drive scale costs every relative Depth Anything V2- and Depth Anything 3 network but one 0.15–0.32 AbsRel on the real benchmark (Section 8.2).

Drives at different times. The evening and dusk drives were added as new sessions to the pose graph of the noon drive, so they must register into the existing map, and they are scored in the map's frame. Single run each.

- With lidar scan odometry, both register at 0.087 and 0.088 m through 18 closures across drives (median error 0.04 m), comparable with driving alone (0.094 and 0.069 m).
- With camera pseudo-scans, closures across drives number 4 (exact depth), 1 (ours; 4 on the chip's output) and 12 (SGBM). Eleven of SGBM's 13 closures are more than 0.3 m off (median cross-drive error 0.71 m), and the network's dusk drive lands 2.06 m off.

The lighting presets are partly a synthetic camera response, because the yard's reconstructed scene barely relights. Real recordings of the same loop at three times of day, planned with a taped start-and-end mark, are the real test.

Table 8.3. Ranging a single prop in the simulator: mean AbsRel over the prop's pixels by distance (sharp frames, one seed, the 9 sharp approaches of 27 rendered; frame counts 129 / 174 / 265 / 351, SGBM 89 / 155 in the two near bins).

Method	0.3–0.75 m	0.75–1.25 m	1.25–2 m	2–3 m
Exact depth (sanity)	0.000	0.000	0.000	0.000
SGBM stereo (aligned)	0.027	0.037	0.044	0.040
Ours (student v5)	0.047	0.078	0.262	0.454
Depth Anything V2 S	0.182	0.197	0.219	0.233
Depth Anything V2 B	0.129	0.146	0.252	0.259
Depth Anything V2 L	0.182	0.202	0.250	0.269
Depth Anything 3 S	0.339	0.128	0.245	0.389
Depth Anything 3 B	0.269	0.135	0.141	0.195
Depth Anything 3 L	0.212	0.086	0.086	0.206
DA3-Mono L	0.405	0.267	0.460	0.507
DA3-Metric L (aligned)	0.109	0.364	0.538	0.572

8.6 Objects at a fixed distance and speed

To isolate how well each method ranges a single object, and whether speed matters, the simulator drives the car straight at one prop (traffic cone, cardboard box or barrel) from 3.8 m at three constant throttles (0.11, 0.44 and 0.73 m/s, measured; the simulated car's nominal top speed is 0.72 m/s), stopping 0.34 m short. The prop's pixels come from an instance-segmentation output of the camera, its exact range from the scene. Each method's estimate is its median over the prop's pixels after the same per-frame alignment as Section 8.2, fitted on 400 held-out pixels that are *not* the prop. Every approach was also rendered with motion blur (33 ms, calibrated against the exact optical flow) and as a sharp twin on identical poses. The renderer's blur is not a physical exposure in shape (a trailing box fits it at 13–15 ms), so a synthetic arm blurs the sharp frames along the exact flow as a cross-check. One seed, one approach per prop and speed; frames within an approach are not independent. Our student is in-distribution here (fine-tuned on exact depth rendered in this yard); every other method is zero-shot.

Distance. At 0.3–0.75 m our student is the most accurate monocular method (AbsRel 0.047 against 0.109 for the next, aligned DA3-Metric), with the in-distribution caveat above. It is best in every one of the nine sharp approaches and their nine sharp re-renders (per-approach AbsRel 0.031–0.066, against 0.057–0.174 for DA3-Metric). At 0.75–1.25 m its lead over Depth Anything 3 L (0.078 against 0.086) is 0.008 on three props, one seed: not a result. Beyond 1.25 m it ranges objects too far (median bias $+0.27$ and $+0.43$); Depth Anything 3 L is best at 1.25–2 m and B at 2–3 m (0.195 against 0.206). Depth Anything V2 reads too near at every distance. DA3-Metric, unaligned, reads $\approx 2.2\times$ too far, the same failure as on the real drive (Section 8.2). SGBM stays within 0.027 unaligned (aligned it is worse, up to 0.044), but lost the barrel on 59 of its 303 near frames.

Speed. In this simulator speed is not a credible test. A blur of one pixel or more appears only within 1.25 m at 0.44 m/s or faster, and paired against the sharp twin it makes no method worse by more than 0.04 (ours $+0.014$, synthetic $+0.038$, at top speed). The floor for

these paired numbers is the renderer itself: a second sharp render of the same poses moves methods by up to ± 0.022 (ours $+0.021$ at 2–3 m), so ours' $+0.014$ is not an effect. Several methods improve, DA3-S most (by 0.068, synthetic 0.075, at 0.73 m/s; by 0.097 at 0.3–0.75 m), plausibly because the blur removes the reconstructed scene's render speckle. The one clear effect is DA3-Metric's raw scale, which drifts $+0.3$ with motion streaks and is removed by alignment. The real protocol of fixed-distance runs at constant throttle remains the test.

Chapter 9

Discussion

9.1 What is new

A depth network the Hailo-8L can actually run. The common assumption is that a foundation model reaches a small NPU through better quantisation. On the Hailo-8L it does not. Attention in int8 is structurally lossy at a few hundred tokens, and neither our build nor the vendor's survives it (Section 3.2). Distilling into a purely convolutional student turns this from a quantisation problem into a training problem, and the result is unusually clean. The int8 student equals its float twin at $r = 0.99995$, so quantisation cost disappears from the error budget entirely. On held-out simulated frames the student is also the most accurate depth network on the chip by a margin that no speed trade explains (0.269 against 0.354–0.530, Table 4.1); on stereo-verified real outdoor points it is not resolvably different from the vendor's Depth Anything V2-S. On the three-matcher set the paired interval bounds any advantage of ours at 0.060 AbsRel, below the simulator's 0.085 (a different metric; in relative terms, 24 %, only the block interval excludes it); without the Depth Anything V2-derived stereo voter it does not (the mean leans our way, -0.061, carried by the vendor model's blow-ups; on log error not resolved). On obstacles, the points above the ground, ours leans toward fewer large errors (log error -0.165, block $p = 0.046$, 0.046–0.14 after Holm depending on the family: post hoc and exploratory), its median is not lower, and with one scale per drive it is worse on the two-matcher set (block $p = 0.040$, equally exploratory; Section 8.2). The simulator's advantage is not shown outdoors. The design choices that make it work are simple: an ImageNet ResNet-34 encoder, a U-Net decoder, a scale-invariant loss, and a sim-ground-truth fine-tune. We see that simplicity as a strength, because it transfers to any integer-only accelerator.

Metric navigation without metric depth. Most depth-based navigation systems either learn metric depth or fuse relative depth with external scale. Ground-contact ranging needs neither. It asks the network only where each column's floor ends, and the camera height does the rest. It is robust to the frame-to-frame scale drift of relative depth, which is the property that matters most on a moving robot. The failure modes it has are geometric and nameable: overhangs, bases lower than the threshold, and surfaces filling a whole row. Each has a measured detector or fix in Chapter 5.

A complete stack in a few watts. Depth (35 ms on the NPU), a 69° metric scan, a hit-count map, global hybrid-A* planning with a safety cascade, pure-pursuit control and a SLAM

front end all fit on a Raspberry Pi 5 at 16–27 Hz. Finke et al. [25] do comparable perception on a Jetson AGX Orin, a computer an order of magnitude more expensive; MonoNav [120] computes depth off-board.

An evaluation that explains itself. Oracle, exact-obstacle, GT-pose and own-pose arms decompose the gap to a perfect score into execution (0.6), perception (1.3) and localisation (0.2 in clean goals, 0.9 in goals reached). Without that decomposition, the many interventions of Table 7.2 would have been tuned blindly.

9.2 Limits, stated plainly

- **Not 10/10.** The deployed system reaches 7.9 of 10 clean with simulator pose and 7.7 on its own pose. The oracle reaches 9.8, so the gap is real and not a property of the benchmark.
- **Low bases.** Cone plates, sign bases and shoes below 5 cm are invisible to the scan. Ground-truth depth would resolve them at a 2 cm threshold; the student's depth does not (Table 5.1).
- **Exploration.** With a $\pm 34^\circ$ camera the car commits to plans through space it has not seen. One scene (crate wall) depends on this more than on anything else.
- **Own pose uses simulated sensors.** The encoder and gyro of Table 6.1 are simulated; the real car has neither yet. Camera-only odometry reaches only 2.6 of 10.
- **One environment.** The simulated yard is a reconstruction of one real yard, and the student was fine-tuned there (on held-out scenes). Real-world validity rests on real-video agreement with the teacher ($r = 0.995$), not on real metric ground truth.
- **No autonomous real drive yet.** The real car has driven tethered under teleoperation. Autonomous drives are gated on a human confirming three runaway failsafes in session. The Pi's camera was not detected when this document was prepared.

9.3 Threats to validity

- *Simulator depth as ground truth.* The renderer's depth for a splat scene has speckle and floaters near the floor. We mitigate this with the obstacle-pixel region (Chapter 4), but absolute AbsRel values should be read as relative comparisons.
- *Seed noise.* About one goal in ten. We use five seeds, and ten where a decision depends on it. Interventions that differ by less than about one goal are reported as neutral.
- *Local brain.* Most later arms ran the Pi's code on the laptop with the Pi's measured latency and planning speed. A replicated Pi arm matched within noise, but small timing effects cannot be ruled out.
- *Metric choice.* Scoring on the car's real footprint is stricter than the simulator's contact circle in some places (front corners) and more lenient in others (sides). We report the footprint metric throughout.

Chapter 10

Conclusion and future work

A foundation-quality monocular depth network can run on a Raspberry Pi 5 with a Hailo-8L, but not as a transformer. Distilled into an attention-free student, it runs at 27 FPS at 288×512, its int8 output equals its float output, and it is the most accurate depth network on the chip. Ranged by ground-contact geometry, its relative depth becomes a metric scan without metric training. On that scan the Pi itself maps, plans with hybrid A*, follows, and localises, at 16–27 Hz. With the real board in a real-time loop through a reconstructed yard, the system reaches the goal without contact in 7.9 of 10 scenes on simulator pose and 7.7 on its own pose (8.2 with the real board in the loop), where the same stack with perfect obstacles reaches 9.2.

10.1 Closing the gap

The measurements point at three concrete next steps, in order of expected value:

1. **Exploration that looks before it commits.** Treat unseen space near a plan as uncertain and steer the camera toward it (perception-aware heading, as in MonoSpheres [97]), instead of planning through it.
2. **Low-structure perception.** Supervise the floor-to-base boundary directly. That could mean a contact-height head trained on rendered ground truth, or semantic detection of cones and people on the same NPU, used conservatively so it does not close narrow gaps.
3. **Temporal depth.** Thin structures flicker from frame to frame. A recurrent or memory-based student, in the spirit of AsyncMDE [83] and Video Depth Anything [12], could stabilise what single frames miss.

10.2 Before the first autonomous drive

1. Fit a rear-axle wheel encoder and a gyro (Table 6.1: 0.08 m median ATE against 0.42 for the camera alone).
2. Calibrate the camera's field of view, height and *pitch*: a 3° downward tilt moves the horizon by about 50 pixels at 720p and changes every range.

3. Check the student's ranges on real ground at tape-measured 0.5, 1.0 and 1.5 m.
4. Do a dry run with the wheels up, then tethered armed runs at 0.25 throttle with the operator deadman in hand, and only after a human has confirmed the three runaway failsafes live in session.

Bibliography

[1] Pratik Agarwal, Gian Diego Tipaldi, Luciano Spinello, Cyrill Stachniss, and Wolfram Burgard. Robust map optimization using dynamic covariance scaling. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 62–69, 2013. doi: 10.1109/icra.2013.6630557.

[2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

[3] Relja Arandjelović, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. NetVLAD: CNN architecture for weakly supervised place recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. arXiv:1511.07247.

[4] Hernán Badino, Uwe Franke, and David Pfeiffer. The stixel world - a compact medium level representation of the 3D-world. In *DAGM Symposium on Pattern Recognition*, pages 51–60, 2009. doi: 10.1007/978-3-642-03798-6_6.

[5] M. Barhoum and A.A. Pyrkin. Monocular depth estimation for 2D mapping of simulated environments. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, 24(1):118–123, 2024. doi: 10.17586/2226-1494-2024-24-1-118-123.

[6] Gabriele Berton and Carlo Masone. MegaLoc: One retrieval to place them all. *arXiv preprint arXiv:2502.17237*, 2025.

[7] Shariq Farooq Bhat, Reiner Birkl, Diana Wofk, Peter Wonka, and Matthias Müller. ZoeDepth: Zero-shot transfer by combining relative and metric depth. *arXiv preprint arXiv:2302.12288*, 2023.

[8] Peter Biber and Tom Duckett. Dynamic maps for long-term operation of mobile service robots. In *Robotics: Science and Systems (RSS)*, 2005. doi: 10.15607/rss.2005.i.003.

[9] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R. Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. In *International Conference on Learning Representations (ICLR)*, 2025.

[10] Johann Borenstein and Yoram Koren. The vector field histogram – fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation*, 7(3):278–288, 1991.

[11] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[12] Sili Chen, Hengkai Guo, Shengnan Zhu, Feihu Zhang, Zilong Huang, Jiashi Feng, and Bingyi Kang. Video depth anything: Consistent depth estimation for super-long videos. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[13] Junda Cheng, Wenjing Liao, Zhipeng Cai, Longliang Liu, Gangwei Xu, Xianqi Wang, Yuzhou Wang, Zikang Yuan, Yong Deng, Jinliang Zang, Yangyang Shi, Jinhui Tang, and Xin Yang. MonSter: Marry monodepth to stereo unleashes power. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6273–6282, 2025. arXiv:2501.08643.

[14] Sungjoon Choi, Qian-Yi Zhou, and Vladlen Koltun. Robust reconstruction of indoor scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5556–5565, 2015. doi: 10.1109/cvpr.2015.7299195.

[15] Winston Churchill and Paul Newman. Experience-based navigation for long-term localisation. *The International Journal of Robotics Research*, 32(14):1645–1661, 2013. doi: 10.1177/0278364913499193.

[16] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments. *arXiv preprint arXiv:1806.08295*, 2018.

[17] R. Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical Report CMU-RI-TR-92-01, Carnegie Mellon University, Robotics Institute, 1992.

[18] Mark Cummins and Paul Newman. FAB-MAP: Probabilistic localization and mapping in the space of appearance. *The International Journal of Robotics Research*, 27(6): 647–665, 2008. doi: 10.1177/0278364908090961.

[19] Frank Dellaert and Michael Kaess. Factor graphs for robot perception. *Foundations and Trends in Robotics*, 6(1-2):1–139, 2017. doi: 10.1561/2300000043.

[20] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. VGGT-Long: Chunk it, loop it, align it – pushing VGGT's limits on kilometer-scale long RGB sequences. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2026. arXiv:2507.16443.

[21] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Path planning for autonomous vehicles in unknown semi-structured environments. *The International Journal of Robotics Research*, 29(5):485–501, 2010.

[22] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[23] Jakob Engel, Thomas Schöps, and Daniel Cremers. LSD-SLAM: Large-scale direct monocular SLAM. In *European Conference on Computer Vision (ECCV)*, pages 834–849, 2014. doi: 10.1007/978-3-319-10605-2_54.

[24] Jakob Engel, Vladlen Koltun, and Daniel Cremers. Direct sparse odometry. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(3), 2018. arXiv:1607.02565.

[25] Jan Finke, Wayne Paul Martis, Adrian Schmelter, Lars Erbach, Christian Jestel, and Marvin Wiedemann. Learning vision-based omnidirectional navigation: A teacher-student approach using monocular depth estimation. *arXiv preprint arXiv:2603.01999*, 2026.

[26] Christian Forster, Matia Pizzoli, and Davide Scaramuzza. SVO: Fast semi-direct monocular visual odometry. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 15–22, 2014. doi: 10.1109/icra.2014.6906584.

[27] Christian Forster, Zichao Zhang, Michael Gassner, Manuel Werlberger, and Davide Scaramuzza. SVO: Semidirect visual odometry for monocular and multicamera systems. *IEEE Transactions on Robotics*, 33(2):249–265, 2017. doi: 10.1109/tro.2016.2623335.

[28] D. Galvez-López and J. D. Tardos. Bags of binary words for fast place recognition in image sequences. *IEEE Transactions on Robotics*, 28(5):1188–1197, 2012. doi: 10.1109/tro.2012.2197158.

[29] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3354–3361, 2012. doi: 10.1109/cvpr.2012.6248074.

[30] Clément Godard, Oisin Mac Aodha, Michael Firman, and Gabriel Brostow. Digging into self-supervised monocular depth estimation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv:1806.01260.

[31] G Grisetti, R Kummerle, C Stachniss, and W Burgard. A tutorial on graph-based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4):31–43, 2010. doi: 10.1109/mits.2010.939925.

[32] Giorgio Grisetti, Cyrill Stachniss, and Wolfram Burgard. Improved techniques for grid mapping with rao-blackwellized particle filters. *IEEE Transactions on Robotics*, 23(1):34–46, 2007. doi: 10.1109/tro.2006.889486.

[33] Hailo Technologies. Hailo model zoo: Zero-shot depth estimation, Hailo-8L. `https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO8L/HAILO8L_zero_shot_depth_estimation.rst`, 2026. Accessed 2026-10-19.

[34] Georg Halmetschlager-Funek, Markus Suchi, Martin Kampel, and Markus Vincze. An empirical evaluation of ten depth cameras: Bias, precision, lateral noise, different lighting conditions and materials, and multiple sensor setups in indoor environments. *IEEE Robotics & Automation Magazine*, 26(1):67–77, 2019. doi: 10.1109/mra.2018.2852795.

[35] Jing He, Haodong Li, Wei Yin, Yixun Liang, Leheng Li, Kaiqiang Zhou, Hongbo Zhang, Bingbing Liu, and Ying-Cong Chen. Lotus: Diffusion-based visual foundation model for high-quality dense prediction. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.18124.

[36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[37] Xiankang He, Dongyan Guo, Hongji Li, Ruibo Li, Ying Cui, and Chi Zhang. Distill any depth: Distillation creates a stronger monocular depth estimator. *arXiv preprint arXiv:2502.19204*, 2025.

[38] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2D LIDAR SLAM. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, 2016. doi: 10.1109/icra.2016.7487258.

[39] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[40] H. Hirschmuller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):328–341, 2008. doi: 10.1109/tpami.2007.1166.

[41] Mu Hu, Wei Yin, et al. Metric3d v2: A versatile monocular geometric foundation model for zero-shot metric depth and surface normal estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[42] Andrey Ignatov, Grigory Malivenko, David Plowman, Samarth Shukla, Radu Timofte, Ziyu Zhang, Yicheng Wang, Zilong Huang, Guozhong Luo, Gang Yu, Bin Fu, Yiran Wang, Xingyi Li, Min Shi, Ke Xian, Zhiguo Cao, Jin-Hua Du, Pei-Lin Wu, Chao Ge, Jiaoyang Yao, Fangwen Tu, Bo Li, Jung Eun Yoo, Kwanggyoon Seo, Jialei Xu, Zhenyu Li, Xianming Liu, Junjun Jiang, Wei-Chi Chen, Shayan Joya, Huanhuan Fan, Zhaobing Kang, Ang Li, Tianpeng Feng, Yang Liu, Chuannan Sheng, Jian Yin, and Fausto T. Benavide. Fast and accurate single-image depth estimation on mobile devices, mobile AI 2021 challenge: Report. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops* (*CVPRW*), pages 2545–2557, 2021. arXiv:2105.08630.

[43] Andrey Ignatov, Grigory Malivenko, Radu Timofte, Lukasz Treszczotko, Xin Chang, Piotr Ksiazek, Michal Lopuszynski, Maciej Pioro, Rafal Rudnicki, Maciej Smyl, Yujie Ma, Zhenyu Li, Zehui Chen, Jialei Xu, Xianming Liu, Junjun Jiang, XueChao Shi, Difan Xu, Yanan Li, Xiaotao Wang, Lei Lei, Ziyu Zhang, Yicheng Wang, Zilong Huang, Guozhong Luo, Gang Yu, Bin Fu, Jiaqi Li, Yiran Wang, Zihao Huang, Zhiguo Cao, Marcos V. Conde, Denis Sapozhnikov, Byeong Hyun Lee, Dongwon Park, Seongmin Hong, Joonhee Lee, Seunggyu Lee, and Se Young Chun. Efficient single-image depth estimation on mobile devices, mobile AI & AIM 2022 challenge: Report. In *European Conference on Computer Vision Workshops* (*ECCVW*), volume 13803 of *LNCS*, pages 71–91, 2022. arXiv:2211.04470.

[44] Sergio Izquierdo and Javier Civera. Optimal transport aggregation for visual place recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2024. arXiv:2311.15937.

[45] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2018.

[46] M. Jalalnezhad, B. T. Sayed, Y. Alotaibi, et al. Real-time vision-based obstacle avoidance for mobile robots using lightweight monocular depth estimation and behavior-driven control. *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, 47: 668, 2025.

[47] Jinwoo Jeon, Dong-Uk Seo, Eungchang Mason Lee, and Hyun Myung. AIM-SLAM: Dense monocular SLAM via adaptive and informative multi-view keyframe prioritization with foundation model. *arXiv preprint arXiv:2603.05097*, 2026.

[48] Hualie Jiang, Zhiqiang Lou, Laiyan Ding, Rui Xu, Minglang Tan, Wenjie Jiang, and Rui Huang. DEFOM-Stereo: Depth foundation model based stereo matching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2501.09466.

[49] Junpeng Jing, Weixun Luo, Ye Mao, and Krystian Mikolajczyk. Lite any stereo: Efficient zero-shot stereo matching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2511.16555.

[50] Junpeng Jing, Ronglai Zuo, Zhelun Shen, Shangchen Zhou, Rolandos Alexandros Potamias, Stefanos Zafeiriou, Krystian Mikolajczyk, and Jiankang Deng. Lite any stereo V2: Faster and stronger efficient zero-shot stereo matching. *arXiv preprint arXiv:2606.24457*, 2026.

[51] Michael Kaess, Hordur Johannsson, Richard Roberts, Viorela Ila, John J Leonard, and Frank Dellaert. iSAM2: Incremental smoothing and mapping using the bayes tree. *The International Journal of Robotics Research*, 31(2):216–235, 2012. doi: 10.1177/0278364911430419.

[52] Bingxin Ke, Anton Obukhov, Shengyu Huang, Nando Metzger, Rodrigo Caye Daudt, and Konrad Schindler. Repurposing diffusion-based image generators for monocular depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[53] Nikhil Keetha, Avneesh Mishra, Jay Karhade, Krishna Murthy Jatavallabhula, Sebastian Scherer, Madhava Krishna, and Sourav Garg. AnyLoc: Towards universal visual place recognition. *IEEE Robotics and Automation Letters*, 2023. arXiv:2308.00688.

[54] Nikhil Keetha, Norman Müller, Johannes Schönberger, Lorenzo Porzi, Yuchen Zhang, Tobias Fischer, Arno Knapitsch, Duncan Zauss, Ethan Weber, Nelson Antunes, Jonathon Luiten, Manuel Lopez-Antequera, Samuel Rota Bulò, Christian Richardt, Deva Ramanan, Sebastian Scherer, and Peter Kontschieder. MapAnything: Universal feed-forward metric 3D reconstruction. In *International Conference on 3D Vision (3DV)*, 2026. arXiv:2509.13414.

[55] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[56] Leonid Keselman, John Iselin Woodfill, Anders Grunnet-Jepsen, and Achintya Bhowmik. Intel RealSense stereoscopic depth cameras. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017. arXiv:1705.05548.

[57] Tobias Koch, Lukas Liebel, Friedrich Fraundorfer, and Marco Körner. Evaluation of CNN-Based single-image depth estimation methods. In *European Conference on Computer Vision Workshops (ECCVW)*, pages 331–348, 2019. doi: 10.1007/978-3-030-11015-4_25.

[58] Stefan Kohlbrecher, Oskar von Stryk, Johannes Meyer, and Uwe Klingauf. A flexible and scalable SLAM system with full 3D motion estimation. In *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 155–160, 2011. doi: 10.1109/ssrr.2011.6106777.

[59] Lingdong Kong, Shaoyuan Xie, Hanjiang Hu, Lai Xing Ng, Benoit R. Cottereau, and Wei Tsang Ooi. RoboDepth: Robust out-of-distribution depth estimation under corruptions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2310.15171.

[60] Lingyu Kong, Ruicheng Li, Ruicheng Wang, Sicheng Xu, Chengtang Yao, Jianfeng Xiang, and Jiaolong Yang. MoGe-3: Fine-detail monocular geometry estimation with self-guided sparse volumetric refinement. *arXiv preprint arXiv:2607.17967*, 2026.

[61] K Konolige, G Grisetti, Rainer Kümmerle, W Burgard, B Limketkai, and R Vincent. Efficient sparse pose adjustment for 2D mapping. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 22–29, 2010. doi: 10.1109/iros.2010.5649043.

[62] Rainer Kümmerle, Bastian Steder, Christian Dornhege, Michael Ruhnke, Giorgio Grisetti, Cyrill Stachniss, and Alexander Kleiner. On measuring the accuracy of SLAM algorithms. *Autonomous Robots*, 27(4):387–407, 2009. doi: 10.1007/s10514-009-9155-6.

[63] Rainer Kummerle, Giorgio Grisetti, Hauke Strasdat, Kurt Konolige, and Wolfram Burgard. g2o: A general framework for graph optimization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3607–3613, 2011. doi: 10.1109/icra.2011.5979949.

[64] Mathieu Labbé and François Michaud. RTABMap as an opensource lidar and visual simultaneous localization and mapping library for largescale and longterm online operation. *Journal of Field Robotics*, 36(2):416–446, 2019. doi: 10.1002/rob.21831.

[65] Yasir Latif, César Cadena, and José Neira. Robust loop closing over time for pose graph SLAM. *The International Journal of Robotics Research*, 32(14):1611–1626, 2013. doi: 10.1177/0278364913498910.

[66] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3D with MASt3R. In *European Conference on Computer Vision (ECCV)*, 2024. arXiv:2406.09756.

[67] Dan Levi, Noa Garnett, and Ethan Fetaya. StixelNet: A deep convolutional network for obstacle detection and road segmentation. In *British Machine Vision Conference (BMVC)*, pages 109.1–109.12, 2015. doi: 10.5244/c.29.109.

[68] Simon D. Levy. BreezySLAM: Simple, efficient, open-source package for simultaneous localization and mapping. `https://github.com/simondlevy/BreezySLAM`, 2014. Software; Python/C port of CoreSLAM (tinySLAM).

[69] Jiankun Li, Peisen Wang, Pengfei Xiong, Tao Cai, Ziwei Yan, Lei Yang, Jiangyu Liu, Haoqiang Fan, and Shuaicheng Liu. Practical stereo matching via cascaded recurrent network with adaptive correlation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2203.11483.

[70] Zhengqi Li, Richard Tucker, Forrester Cole, Qianqian Wang, Linyi Jin, Vickie Ye, Angjoo Kanazawa, Aleksander Holynski, and Noah Snavely. MegaSaM: Accurate, fast, and robust structure and motion from casual dynamic videos. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2412.04463.

[71] Zhikai Li, Junrui Xiao, Lianwei Yang, and Qingyi Gu. Repq-vit: Scale reparameterization for post-training quantization of vision transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[72] Haotong Lin, Sida Peng, Jingxiao Chen, Songyou Peng, Jiaming Sun, Minghuan Liu, Hujun Bao, Jiashi Feng, Xiaowei Zhou, and Bingyi Kang. Prompting depth anything for 4K resolution accurate metric depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2412.14015.

[73] Haotong Lin, Sili Chen, Jun Hao Liew, et al. Depth anything 3: Recovering the visual space from any views. In *International Conference on Learning Representations* (*ICLR*), *oral*, 2026. arXiv:2511.10647.

[74] Yang Lin, Tianyu Zhang, Peiqin Sun, Zheng Li, and Shuchang Zhou. Fq-vit: Post-training quantization for fully quantized vision transformer. In *International Joint Conference on Artificial Intelligence* (*IJCAI*), 2022.

[75] Lahav Lipson, Zachary Teed, and Jia Deng. RAFT-Stereo: Multilevel recurrent field transforms for stereo matching. In *International Conference on 3D Vision* (*3DV*), 2021. arXiv:2109.07547.

[76] Lahav Lipson, Zachary Teed, and Jia Deng. Deep patch visual SLAM. In *European Conference on Computer Vision* (*ECCV*), volume 15060 of *LNCS*, pages 424–440, 2024. arXiv:2408.01654.

[77] Muxin Liu, Xiaoyang Lyu, Yang-Tian Sun, Yi-Hua Huang, Ziyi Yang, Peng Dai, and Xiaojuan Qi. Monocular depth estimation from a single image: Progress and opportunities. *arXiv preprint arXiv:2609.01172* (*accepted to Computational Visual Media*), 2026.

[78] Yuzheng Liu, Siyan Dong, Shuzhe Wang, Yingda Yin, Yanchao Yang, Qingnan Fan, and Baoquan Chen. SLAM3R: Real-time dense scene reconstruction from monocular RGB videos. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2412.09401.

[79] Shing Yan Loo, Ali Jahani Amiri, Syamsiah Mashohor, Sai Hong Tang, and Hong Zhang. CNN-SVO: Improving the mapping in semi-direct visual odometry using single-image depth prediction. In *IEEE International Conference on Robotics and Automation* (*ICRA*), pages 5218–5223, 2019. doi: 10.1109/icra.2019.8794425.

[80] Stephanie Lowry, Niko Sünderhauf, Paul Newman, John J. Leonard, David Cox, Peter Corke, and Michael J. Milford. Visual place recognition: A survey. *IEEE Transactions on Robotics*, 32(1):1–19, 2016. doi: 10.1109/tro.2015.2496823.

[81] F. Lu and E. Milios. Globally consistent range scan alignment for environment mapping. *Autonomous Robots*, 4(4):333–349, 1997. doi: 10.1023/a:1008854305733.

[82] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence* (*IJCAI*), 1981.

[83] Lianjie Ma, Yuquan Li, Bingzheng Jiang, Ziming Zhong, Han Ding, and Lijun Zhu. Asyncmde: Real-time monocular depth estimation via asynchronous spatial memory. *arXiv preprint arXiv:2603.10438*, 2026.

[84] Steve Macenski and Ivona Jambrecic. SLAM toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021. doi: 10.21105/joss.02783.

[85] Will Maddern, Geoffrey Pascoe, Chris Linegar, and Paul Newman. 1 year, 1000 km: The oxford RobotCar dataset. *The International Journal of Robotics Research*, 36(1):3–15, 2017. doi: 10.1177/0278364916679498.

[86] Dominic Maggio and Luca Carlone. VGGT-SLAM 2.0: Real-time dense feed-forward scene reconstruction. *arXiv preprint arXiv:2601.19887*, 2026.

[87] Dominic Maggio, Hyungtae Lim, and Luca Carlone. VGGT-SLAM: Dense RGB SLAM optimized on the SL(4) manifold. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2025. arXiv:2505.12549.

[88] Avilasha Mandal, Rajesh Kumar, Sudarshan Sunil Harithas, and Chetan Arora. VGGT-SLAM++. *arXiv preprint arXiv:2604.06830*, 2026. CVPR 2026 Workshop (VOCVALC).

[89] Joshua G. Mangelson, Derrick Dominic, Ryan M. Eustice, and Ram Vasudevan. Pairwise consistent measurement set maximization for robust multi-robot map merging. In *IEEE International Conference on Robotics and Automation* (*ICRA*), pages 2916–2923, 2018. doi: 10.1109/icra.2018.8460217.

[90] Rémi Marsal, Quentin Picard, Adrien Poiré, Sébastien Kerbourc'h, Thibault Toralba, Clément Yver, Alexandre Chapoutot, and David Filliat. An open-source LiDAR and monocular off-road autonomous navigation stack. *arXiv preprint arXiv:2604.03096*, 2026.

[91] Colin McManus, Winston Churchill, Will Maddern, Alexander D. Stewart, and Paul Newman. Shady dealings: Robust, long-term visual localisation using illumination invariance. In *IEEE International Conference on Robotics and Automation* (*ICRA*), pages 901–906, 2014. doi: 10.1109/icra.2014.6906961.

[92] Jeff Michels, Ashutosh Saxena, and Andrew Y. Ng. High speed obstacle avoidance using monocular vision and reinforcement learning. In *International Conference on Machine Learning* (*ICML*), pages 593–600, 2005. doi: 10.1145/1102351.1102426.

[93] Michael J. Milford and Gordon. F. Wyeth. SeqSLAM: Visual route-based navigation for sunny summer days and stormy winter nights. In *IEEE International Conference on Robotics and Automation* (*ICRA*), pages 1643–1649, 2012. doi: 10.1109/icra.2012.6224623.

[94] H. Moravec and A. Elfes. High resolution maps from wide angle sonar. In *IEEE International Conference on Robotics and Automation* (*ICRA*), volume 2, pages 116–121, None. doi: 10.1109/robot.1985.1087316.

[95] Raul Mur-Artal, J. M. M. Montiel, and Juan D. Tardos. ORB-SLAM: a versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5), 2015. arXiv:1502.00956.

[96] Riku Murai, Eric Dexheimer, and Andrew J. Davison. Mast3r-slam: Real-time dense slam with 3d reconstruction priors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025.

[97] Tomáš Musil, Matěj Petrlík, and Martin Saska. Monospheres: Large-scale monocular slam-based uav exploration through perception-coupled mapping and planning. *IEEE Robotics and Automation Letters* (*arXiv:2511.17299*), 2025.

[98] NVIDIA et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[99] E.B. Olson. Real-time correlative scan matching. In *IEEE International Conference on Robotics and Automation* (*ICRA*), pages 4387–4393, 2009. doi: 10.1109/robot.2009.5152375.

[100] Edwin Olson and Pratik Agarwal. Inference on networks of mixtures for robust robot mapping. In *Robotics: Science and Systems (RSS)*, 2012. doi: 10.15607/rss.2012.viii.040.

[101] Luigi Piccinelli, Yung-Hsu Yang, Christos Sakaridis, Mattia Segu, Siyuan Li, Luc Van Gool, and Fisher Yu. Unidepth: Universal monocular metric depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[102] Luigi Piccinelli, Christos Sakaridis, Mattia Segu, Yung-Hsu Yang, Siyuan Li, Wim Abbeloos, and Luc Van Gool. UniK3D: Universal camera monocular 3D estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2503.16591.

[103] Luigi Piccinelli, Christos Sakaridis, Yung-Hsu Yang, Mattia Segu, Siyuan Li, Wim Abbeloos, and Luc Van Gool. UniDepthV2: Universal monocular metric depth estimation made simpler. *arXiv preprint arXiv:2502.20110*, 2025.

[104] Matteo Poggi, Filippo Aleotti, Fabio Tosi, and Stefano Mattoccia. Towards real-time unsupervised monocular depth estimation on CPU. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018. arXiv:1806.11430.

[105] Nishant Pushparaju, Vivek Mattam, and Aliasghar Arab. Bootstrap perception under hardware depth failure for indoor robot navigation. *arXiv preprint arXiv:2603.28890*, 2026.

[106] Tong Qin, Peiliang Li, and Shaojie Shen. VINS-Mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4), 2018. arXiv:1708.03852.

[107] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[108] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3): 1623–1637, 2022.

[109] Raspberry Pi Ltd. Raspberry pi ai kit: product brief. `https://datasheets.raspberrypi.com/ai-kit/raspberry-pi-ai-kit-product-brief.pdf`, 2024. Hailo-8L M.2 module, 13 TOPS, published June 2024.

[110] RealSense. RealSense D400 series product family datasheet. `https://www.realsenseai.com/wp-content/uploads/2026/03/RealSense-D400-Series-Datasheet-Mar-2026.pdf`, 2026. Revision March 2026.

[111] RealSense. Tuning depth cameras for best performance. `https://dev.realsenseai.com/docs/tuning-depth-cameras-for-best-performance`, 2026. Vendor white paper, last updated 2026-02-04; accessed 2026-10-19.

[112] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[113] ROS perception. depthimage_to_laserscan. `https://github.com/ros-perception/depthimage_to_laserscan`, 2012. ROS package; software.

[114] Pascal Roth, Julian Nubert, Fan Yang, Mayank Mittal, and Marco Hutter. Viplanner: Visual semantic imperative learning for local navigation. In *IEEE International Conference on Robotics and Automation* (*ICRA*), 2024.

[115] Simão Ruás Branco. Real-time depth estimation in limited hardware environments: A performance and resource trade-off analysis of SOTA methods. M.sc. thesis (robotics), Aalborg University, 2026. Submitted 2026-06-03. `https://projekter.aau.dk/`.

[116] Lukas Rustler, Vojtech Volprecht, and Matej Hoffmann. Empirical comparison of four stereoscopic depth sensing cameras for robotics applications. *IEEE Access*, 13:67564–67577, 2025. arXiv:2501.07421.

[117] Torsten Sattler, Will Maddern, Carl Toft, Akihiko Torii, Lars Hammarstrand, Erik Stenborg, Daniel Safari, Masatoshi Okutomi, Marc Pollefeys, Josef Sivic, Fredrik Kahl, and Tomas Pajdla. Benchmarking 6DOF outdoor visual localization in changing conditions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2018. arXiv:1707.09092.

[118] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. Vint: A foundation model for visual navigation. In *Conference on Robot Learning* (*CoRL*), 2023.

[119] Basant Sharma, Prajyot Jadhav, Pranjal Paul, K. Madhava Krishna, and Arun Kumar Singh. Monompc: Monocular vision based navigation with learned collision model and risk-aware model predictive control. *IEEE Robotics and Automation Letters* (*arXiv:2508.07387*), 2026.

[120] Nathaniel Simon and Anirudha Majumdar. Mononav: Mav navigation via monocular depth estimation and reconstruction. In *International Symposium on Experimental Robotics* (*ISER*), 2023.

[121] Shiyu Song and Manmohan Chandraker. Robust scale estimation in real-time monocular SFM for autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pages 1566–1573, 2014. doi: 10.1109/cvpr.2014.203.

[122] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In *IEEE International Conference on Robotics and Automation* (*ICRA*), 2024.

[123] Bruno Steux and Oussama El Hamzaoui. tinySLAM: A SLAM algorithm in less than 200 lines c-language program. In *International Conference on Control, Automation, Robotics and Vision* (*ICARCV*), pages 1975–1979, 2010. doi: 10.1109/icarcv.2010.5707402.

[124] Jrgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems* (*IROS*), pages 573–580, 2012. doi: 10.1109/iros.2012.6385773.

[125] Boyuan Sun, Modi Jin, Bowen Yin, and Qibin Hou. Depth anything at any condition. *arXiv preprint arXiv:2507.01634*, 2025.

[126] Libo Sun, Jia-Wang Bian, et al. Sc-depthv3: Robust self-supervised monocular depth estimation for dynamic scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(1):497–508, 2024.

[127] Niko Sunderhauf and Peter Protzel. Switchable constraints for robust pose graph SLAM. In *IEEE/RSJ International Conference on Intelligent Robots and Systems* (*IROS*), pages 1879–1884, 2012. doi: 10.1109/iros.2012.6385590.

[128] Keisuke Tateno, Federico Tombari, Iro Laina, and Nassir Navab. CNN-SLAM: Real-time dense monocular SLAM with learned depth prediction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2017. arXiv:1704.03489.

[129] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2021.

[130] Zachary Teed, Lahav Lipson, and Jia Deng. Deep patch visual odometry. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023.

[131] Fabio Tosi, Luca Bartolomei, Matteo Poggi, and Stefano Mattoccia. ZipDepth: Bringing lightweight zero-shot monocular depth anywhere, on any device. In *European Conference on Computer Vision* (*ECCV*), 2026. arXiv:2607.08771.

[132] Jonas Uhrig, Nick Schneider, Lukas Schneider, Uwe Franke, Thomas Brox, and Andreas Geiger. Sparsity invariant CNNs. In *International Conference on 3D Vision* (*3DV*), 2017. arXiv:1708.06500.

[133] Iwan Ulrich and Illah Nourbakhsh. Appearance-based obstacle detection with monocular color vision. In *AAAI Conference on Artificial Intelligence* (*AAAI*), pages 866–871, 2000.

[134] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4): 376–380, 1991. doi: 10.1109/34.88573.

[135] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2503.11651.

[136] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A. Efros, and Angjoo Kanazawa. Continuous 3D perception model with persistent state. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2501.12387.

[137] Ruicheng Wang, Sicheng Xu, Cassie Dai, Jianfeng Xiang, Yu Deng, Xin Tong, and Jiaolong Yang. MoGe: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2410.19115.

[138] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2025. arXiv:2507.02546.

[139] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D vision made easy. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2024. arXiv:2312.14132.

[140] Yan Wang, Wei-Lun Chao, Divyansh Garg, Bharath Hariharan, Mark Campbell, and Kilian Q. Weinberger. Pseudo-LiDAR from visual depth estimation: Bridging the gap in 3D object detection for autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2019. arXiv:1812.07179.

[141] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3: Permutation-equivariant visual geometry learning. In *International Conference on Learning Representations* (*ICLR*), 2026. arXiv:2507.13347.

[142] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. FoundationStereo: Zero-shot stereo matching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025. arXiv:2501.09898.

[143] Bowen Wen, Shaurya Dewan, and Stan Birchfield. Fast-FoundationStereo: Real-time zero-shot stereo matching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pages 7513–7524, 2026. arXiv:2512.11130.

[144] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. Fastdepth: Fast monocular depth estimation on embedded systems. In *IEEE International Conference on Robotics and Automation* (*ICRA*), 2019.

[145] Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moenne-Loccoz, and Zan Gojcic. 3dgut: Enabling distorted cameras and secondary rays in gaussian splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2025.

[146] Gangwei Xu, Xianqi Wang, Xiaohuan Ding, and Xin Yang. Iterative geometry encoding volume for stereo matching. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2023. arXiv:2303.06615.

[147] Gangwei Xu, Xianqi Wang, Zhaoxing Zhang, Junda Cheng, Chunyuan Liao, and Xin Yang. IGEV++: Iterative multi-range geometry encoding volumes for stereo matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025. arXiv:2409.00638.

[148] Zhaoming Xu, Chan-Wei Hu, Kuan-Ru Huang, Zihao Zhu, Renjie Li, Yang Zhou, and Zhengzhong Tu. Weather-conditioned depth anything. *arXiv preprint arXiv:2609.04827*, 2026.

[149] Feng Xue, Wu Chen, Mingshuai Zhao, Guofeng Zhong, Anlong Ming, Haozhe Wang, Dianqiao Lei, Zhaowen Lin, Haiyang Zhang, and Nicu Sebe. DepthART: Scaling foundation monocular depth to tiny models. In *ACM International Conference on Multimedia* (*ACM MM*), 2026. arXiv:2607.17099; not the unrelated autoregressive DepthART, arXiv:2409.15010.

[150] Fan Yang, Chen Wang, Cesar Cadena, and Marco Hutter. iplanner: Imperative path planning. In *Robotics: Science and Systems* (*RSS*), 2023.

[151] Heng Yang, Pasquale Antonante, Vasileios Tzoumas, and Luca Carlone. Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection. *IEEE Robotics and Automation Letters*, 5(2), 2020. arXiv:1909.08605.

[152] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2024.

[153] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2024.

[154] Nan Yang, Rui Wang, Jörg Stückler, and Daniel Cremers. Deep virtual stereo odometry: Leveraging deep depth prediction for monocular direct sparse odometry. In *European Conference on Computer Vision (ECCV)*, 2018. arXiv:1807.02570.

[155] Nan Yang, Lukas von Stumberg, Rui Wang, and Daniel Cremers. D3VO: Deep depth, deep pose and deep uncertainty for monocular visual odometry. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:2003.01060.

[156] Yurong You, Yan Wang, Wei-Lun Chao, Divyansh Garg, Geoff Pleiss, Bharath Hariharan, Mark Campbell, and Kilian Q. Weinberger. Pseudo-LiDAR++: Accurate depth for 3D object detection in autonomous driving. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1906.06310.

[157] Zhihang Yuan, Chenhao Xue, Yiqi Chen, Qiang Wu, and Guangyu Sun. Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization. In *European Conference on Computer Vision (ECCV)*, 2022.

[158] Ramin Zabih and John Woodfill. Non-parametric local transforms for computing visual correspondence. In *European Conference on Computer Vision (ECCV)*, pages 151–158, 1994. doi: 10.1007/bfb0028345.

[159] Ning Zhang, Francesco Nex, George Vosselman, and Norman Kerle. Lite-mono: A lightweight cnn and transformer architecture for self-supervised monocular depth estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[160] Yuhao Zhang, Yifu Tao, Frank Dellaert, and Maurice Fallon. ScaRF-SLAM: Scale-consistent reconstruction with feed-forward models and classical visual SLAM. *IEEE Robotics and Automation Letters*, 11(11):12519–12526, 2026. arXiv:2606.00307.

[161] Zichao Zhang and Davide Scaramuzza. A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7244–7251, 2018. doi: 10.1109/iros.2018.8593941.

[162] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3D: A modern library for 3D data processing. *arXiv preprint arXiv:1801.09847*, 2018.

Appendix A

Reproduction

All code is in the project repository (`itaim18/DANIEL`). Paths below are relative to its root.

A.1 Depth student

`board/pi5/student_depth.py`	frames / label / train / eval / export; `--floor-w` for v5
`board/pi5/student_hailo.alls`	Hailo compile recipe (normalisation, optimisation level 1)
`board/pi5/edge_depth.py`	on-device inference for every HEF (centre crop at the model's aspect)
`experiments/2026-09-30_student_depth/`	weights, HEFs, training logs, README

A.2 Benchmark (Chapter 4)

All in `experiments/2026-10-02_thesis_bench/`:

`frames.json`	the 400 held-out benchmark frames
`bench_baselines.py`	GPU predictions: Depth Anything V2 S/B/L and Depth Anything 3
`bench_students.py`	GPU predictions of the students (the chip's float twin)
`chipbench.py`	on the Pi: every HEF on the same frames, outputs + timing
`score.py`	alignment, regions, metrics → `scores.json`
`fig_*.py`, `make_demo.py`	every figure of this thesis and the 1080p demo video
`gpu_timing.py`, `cpubench.py`	float-GPU and Pi-CPU latency references
`hitl_data.py`, `make_reel.py`	per-seed loop results; labelled reels of a run
`report/build.py`	the web report, from the same JSON files

Tables: `thesis/make_tables.py` generates every table from the JSON files (no hand-copied numbers).

A.3 Navigation stack and HITL

`board/pi5/avoid.py`	ground-contact scan, floor model, detectors, reactive layer (self-tested)
`board/pi5/pathplan.py`	map, hybrid A*, cascade, follower, Navigator (self-tested)
`board/pi5/slam2d.py`	ground VO, SLAM, wheel odometry, slip guard (self-tested)
`board/pi5/hitl_pi.py`	the robot computer's server (Hailo or local torch backend)
`board/pi5/avoid_drive.py`	the onboard runner for the real car (dry-run by default)
`wheeled_lab/rl/hitl_sim.py`	the Isaac Sim side of the loop
`experiments/2026-09-29_hitl/`	scenarios, per-episode logs, summaries
`docs/findings.md`	the dated record of every finding, including retractions

A.4 Commands

```
# on the Pi: serve depth + planning for the simulator
python3 hitl_pi.py            # default HEF: ~/models/dav2student_v5_288x512.hef
# on the workstation: one arm, real time, own pose with wheel odometry
./run_headless.sh log.txt hitl_sim.py --brain pi --nav --slam --realtime \
    --esc-gate --cam-h 0.133 --stuck-s 35 --odo imu --seed 1
# benchmark + figures + thesis
python3 score.py && python3 fig_bench.py && cd thesis && tectonic main.tex
```